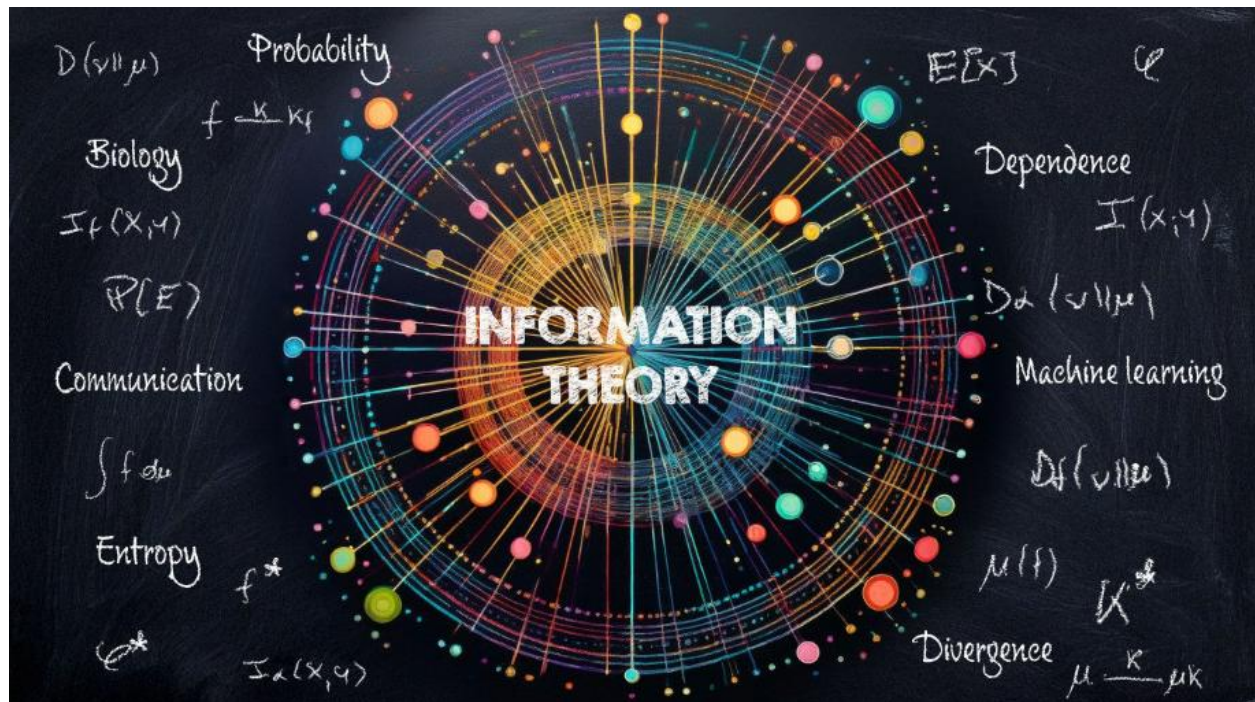


INFORMATION THEORY TOOLKIT



УНИВЕРЗИТЕТ „ГОЦЕ ДЕЛЧЕВ”- ШТИП
ФАКУЛТЕТ ЗА ИНФОРМАТИКА
Компјутерско инженерство и технологии

СЕМИНАРСКА РАБОТА
Алатка за теорија на информации



Ментор,
Проф.Др. Наташа Стојковиќ

Студент,
Мерсие Адемова
број на индекс:102856

Штип, јуни 2026

Содржина

1. Вовед	4
2. Цели на проектот	5
3. Теоретска основа	6
4. Архитектура и имплементација на апликацијата	8
4.1. Модул за пресметка на ентропија (entropy.py)	9
4.2. Алгоритам на Huffman (huffman.py)	12
4.3. Алгоритам на Shannon – Fano (shannon_fano.py)	21
4.4. Модул за анализа на кодови (code_checker.py)	27
4.5. Модул за анализа на комуникациски канал (channel.py)	32
4.6. Модул за компресија на податоци (compression.py)	36
4.7. Модул за споредба (comparison.py)	39
5. Тестирање и анализа на резултатите (Case Study)	42
5.1. Тест анализа на ентропија	43
5.2. Анализа на резултатите од Huffman кодирањето	45
5.3. Анализа на резултатите од Shannon – Fano кодирањето	48
5.4. Тест анализа на кодни зборови	50
5.5. Тест анализа на комуникациски канал	52
5.6. Тест анализа на модулот за компресија	56
5.7. Тест анализа на модулот за споредба меѓу двата анализирани алгоритми	58
6. Можности за понатамошен развој на апликацијата	59
7. Заклучок	61
8. Користена литература	62

Information theory toolkit

1. Вовед

Теоријата на информации е научна дисциплина која се занимава со квантитативно проучување на информацијата, нејзиното претставување, пренос и обработка. Таа ги поставува математичките основи за тоа како информацијата може да се мери, кодира и ефективно да се пренесува преку комуникациски канали, особено во присуство на шум и ограничени ресурси.

Основен концепт во Теоријата на информации е ентропијата, која претставува мерка за неизвесноста или количината на информација во еден извор. Покрај тоа, дисциплината опфаќа и методи за оптимално кодирање на податоци, како што се *Huffman* и *Shannon – Fano* кодовите, кои овозможуваат компресија на податоци со минимална загуба на информации.

Важноста на Теоријата на информации е огромна во современите дигитални системи. Таа претставува основа за развој на сите форми на дигитална комуникација, вклучувајќи интернет протоколи, компресија на податоци (како zip и mp3), мобилни комуникации, криптографија и машинско учење. Без овие концепти, ефикасниот пренос и складирање на информации во денешните системи би бил невозможен.

Развиената апликација “Information theory toolkit“, претставува практична имплементација на основните концепти од Теоријата на информации. Апликацијата овозможува пресметка на ентропија, генерирање на оптимални кодови (*Huffman* и *Shannon – Fano*), анализа на својства на кодови (како префикс услов и *Kraft* неравенство), анализа на комуникациски канали преку пресметка на заемна информација, како и симулација на компресија на податоци.

Со оваа алатка, теоретските концепти стануваат визуелно и практично разбирливи, овозможувајќи подобро разбирање на начинот на кој информацијата се обработува и оптимизира во реални системи.

2. Цели на проектот

Главната цел на овој проект е практична имплементација и анализа на основните концепти од Теоријата на информации преку развој на интерактивна апликација која овозможува визуелизација и споредба на методи за кодирање и пренос на информации.

Проектот има за цел да ги поврзе теоретските поими со нивната практична примена, со што се овозможува подобро разбирање на начинот на кој информацијата се мери, кодира, компресира и пренесува во дигиталните системи.

Конкретните цели на проектот се:

- Анализа на информациски извори преку пресметка на ентропија и фреквенција на симболи;
- Имплементација на методи за оптимално кодирање на податоци, вклучувајќи *Huffman* и *Shannon – Fano* алгоритми;
- Проверка на својствата на кодовите, како што се префикс својство и Kraft неравенство;
- Анализа на комуникациски канал преку пресметка на условна ентропија и заемна информација;
- Симулација и анализа на компресија на податоци и споредба на ефикасноста на различни методи за кодирање.

3. Теоретска основа

Во овој дел од проектот се разгледуваат и дефинираат основните концепти од Теоријата на информации кои претставуваат теоретска основа за развојот и функционирањето на имплементираната апликација.

Целта на ова поглавје е да се претстават математички модели и формули кои се користат за мерење, анализа и обработка на информацијата, како и нивната улога во процесите на кодирање, компресија и пренос на податоци.

Посебен акцент е ставен на поимите ентропија, сопствена информација, просечна должина на код, ефикасност на код, како и на условна и заедничка ентропија, заемна информација и релативна ентропија. Дополнително, се разгледува и Kraft неравенство како важен услов за постоење на префикс кодови.

Теоријата на информации се базира на математички модели кои ја опишуваат количината, структурата и преносот на информацијата. Во продолжение се дадени основните поими кои се користат во овој проект.

Ентропијата претставува мерка за неизвесноста или количината на информација во еден случаен извор:

$$H(X) = - \sum p(x) \log_2 p(x)$$

каде:

- $p(x)$ е веројатноста на појавување на симболот x .

Ентропијата ја мери просечната количина на информација по симбол.

Сопствената информација на еден настан ја мери “изненадувачката вредност” на тој настан:

$$I(X) = -\log_2 p(x)$$

Ретки настани носат поголема информација, додека честите настани носат помала информација.

Просечната должина на кодот претставува очекуван број на битови потребни за кодирање на симболите:

$$L = \sum p_i n_i$$

каде:

- p_i е веројатноста на симболот;
- n_i е должината на кодот за тој симбол.

Ефикасноста покажува колку блиску е кодот до теоретски оптималната граница:

$$\eta = \frac{H(X)}{L}$$

- Вредност блиска до 1 значи висока ефикасност.

Kraft неравенството е услов за постоење на префикс код:

$$\sum 2^{-n_i} \leq 1$$

каде:

- n_i е должината на кодните зборови.

Условната ентропија ја мери неизвесноста на Y ако X е познат:

$$H(Y|X) = - \sum p(x, y) \log_2 p(y|x)$$

Заемната ентропија ја мери вкупната неизвесност на две случајни променливи:

$$H(X, Y) = - \sum p(x, y) \log_2 p(x, y)$$

Заемната информација мери колку информации споделуваат две променливи:

$$I(X, Y) = H(X) - H(X|Y)$$

Односно,

$$I(X, Y) = H(X) + H(Y) - H(X, Y)$$

Релативната ентропија ја мери разликата меѓу две распределби:

$$D(P||Q) = \sum p(x) \log_2 \frac{p(x)}{q(x)}$$

4. Архитектура и имплементација на апликацијата

Развиената апликација „Information Theory Toolkit“ претставува интерактивна софтверска алатка наменета за практична демонстрација и анализа на основните концепти од Теоријата на информации. Апликацијата е развиена во Python и овозможува визуелизација, пресметка и споредба на различни методи за кодирање и обработка на информации.

Апликацијата е организирана во повеќе функционални модули, од кои секој покрива одредена област од теоријата. Овие модули вклучуваат пресметка на ентропија, имплементација на алгоритмите *Huffman* и *Shannon – Fano* за оптимално кодирање, проверка на својствата на кодовите, анализа на комуникациски канал, како и симулација на компресија на податоци.

Корисникот има можност да внесе текстуални податоци, по што апликацијата автоматски ги анализира и ги претвора во соодветни кодови. Дополнително, резултатите се прикажуваат во форма на текстуални извештаи и графички визуелизации, што овозможува полесно разбирање на разликите помеѓу различните методи.

Посебен акцент е ставен на споредбата на ефикасноста на *Huffman* и *Shannon – Fano* кодирањето, како и на визуелна репрезентација на фреквенциите и структурата на кодовите.

4.1. Модул за пресметка на ентропија (entropy.py)

Модулот за пресметка на ентропија претставува основен дел од апликацијата и има за цел да ја демонстрира концепцијата за информациска несигурност кај дискретен извор. Во овој модул корисникот внесува множество на веројатности кои го опишуваат појавувањето на одредени симболи, а апликацијата врз основа на тие податоци ја пресметува ентропијата на изворот.


```

def create_entropy_tab(parent):
    def calculate_entropy():
        try:
            probs = list(
                map(float,
                    entropy_entry.get().split())
            )
            if abs(sum(probs) - 1.0) > 0.001:
                entropy_result.config(
                    text="Грешка: Збирот мора да биде 1."
                )
                return
            H = 0
            for p in probs:
                if p > 0:
                    H -= p * math.log2(p)
            n = len(probs)
            max_entropy = math.log2(n)
            redundancy = 1 - (H / max_entropy)
            result = (
                f"Ентропија H(X) = {H:.4f} bits\n"
                f"Максимална ентропија = {max_entropy:.4f} bits\n"
                f"Редундантност = {redundancy:.4f}\n"
                f"Број на симболи = {n}"
            )

```

4.1. Функција за пресметување на ентропијата на изворот

Функцијата *calculate_entropy()*, прикажана на слика 4.1., е централниот дел на модулот за анализа на информациски извор, чија задача е да ја пресмета ентропијата врз основа на внесена распределба на веројатности.

Функцијата започнува со читање на податоците внесени од корисникот преку графичкиот интерфејс. Внесените вредности се претвараат во листа од реални броеви кои ги претставуваат веројатностите на појавување на поединечните симболи од изворот. Бидејќи ентропијата е дефинирана врз веројатносна распределба, неопходно е да се провери дали внесените вредности формираат валидна распределба.

За таа цел се пресметува нивниот збир и се проверува дали е еднаков на 1. Доколку условот не е исполнет, функцијата ја прекинува понатамошната обработка и прикажува порака за грешка. Оваа проверка е важна бидејќи сите понатамошни пресметки во Теоријата на информации претпоставуваат дека работиме со валидна распределба на веројатности.

По успешна валидација, се пристапува кон пресметка на ентропијата. За секоја веројатност поголема од нула се пресметува нејзиниот придонес во вкупната ентропија според Шеноновата формула:

$$H(X) = - \sum p(x) \log_2 p(x)$$

Во оваа формула, логаритамот со база два овозможува резултатите да биде изразен во битови, што е стандардна мерна единица во Теоријата на информации. Секој член од сумата ја претставува количината на информација поврзана со појавувањето на одреден симбол, пондерирана со неговата веројатност.

Откако ќе се добие вредноста на ентропијата, функцијата ја пресметува и максималната можна ентропија за дадениот број симболи. Максималната ентропија се јавува кога сите симболи се подеднакво веројатни и се пресметува според формулата:

$$H_{max} = \log_2(n)$$

каде што n го претставува бројот на симболи во изворот.

Врз основа на реалната и максималната ентропија, функцијата ја пресметува и редундантноста на изворот:

$$R = 1 - \frac{H(X)}{H_{max}}$$

Редундантноста претставува мерка за предвидливоста на изворот и покажува колкав дел од неговата структура не носи нова информација. Извор со висока редундантност е полесен за компресија, додека извор со ентропија блиска до максималната има помал потенцијал за дополнително намалување на количината на податоци.

На крајот, функцијата ги форматира добиените резултати и ги прикажува во корисничкиот интерфејс. Корисникот добива информации за пресметаната ентропија, максималната ентропија, редундантноста и бројот на симболи во анализираниот извор.

Графичкиот интерфејс на модулот е реализиран преку *Tkinter* и се состои од едноставно поле за внесување, каде што веројатностите се внесуваат како низа од вредности одделени со празно место. Дополнително, постои копче со кое се иницира пресметката и поле во кое се прикажуваат резултатите.

4.2. Алгоритам на Huffman ([huffman.py](#))

Huffman модулот има за цел имплементација на алгоритмот за оптимално префикс – кодирање познато како *Huffman* кодирање. Овој алгоритам се користи за компресија на податоци преку доделување на бинарни кодови за променлива должина на симболите, во зависност од нивната фреквенција на појавување.

Основната идеја на *Huffman* кодирањето е почестите симболи да добијат пократки кодови, додека поретките симболи добиваат подолги кодови. На тој начин се намалува вкупниот број на битови потребни за претставување на една порака, што директно води кон компресија на податоците.

```
class Node:
    def __init__(self, symbol, freq):
        self.symbol = symbol
        self.freq = freq
        self.left = None
        self.right = None
    def __lt__(self, other):
        return self.freq < other.freq
```

4.2. Класа *Node*

Класата *Node*, прикажана на слика 4.2., е основен елемент во имплементацијата на *Huffman* алгоритмот и се користи за претставување на јазлите во *Huffman* дрвото. Бидејќи алгоритмот работи со бинарно дрво, потребно е да постои структура која ќе ги чува информациите за секој симбол, неговата фреквенција и врските со останатите јазли.

Оваа класа ги енкапсулира сите податоци потребни за конструкција на дрвото и овозможува негово постепено градење преку спојување на јазлите со најмали фреквенции.

При креирање на нов објект од оваа класа се иницијализираат неколку атрибути. Атрибутот *symbol* го претставува симболот од азбуката кој се кодира, додека атрибутот *freq* ја содржи неговата фреквенција на појавување во анализираниот текст. Дополнително се дефинираат атрибутите *left* и *right*, кои претставуваат покажувачи кон левото и десното поддрво и овозможуваат формирање на бинарна дрвовидна структура.

Кај листовите на дрвото, атрибутот *symbol* содржи конкретен знак од текстот, додека кај внатрешните јазли неговата вредност е празна (*none*), бидејќи тие не претставуваат симболи туку резултат од спојување на два постоечки јазли. Нивната фреквенција се добива како збир од фреквенциите на нивните деца.

Методот *__lt__()* го дефинира начинот на споредување на два јазли и овозможува нивно правилно користење во приоритетна редица (*priority queue*) и истиот враќа логичка вредност врз основа на споредба на фреквенциите на два јазли. Ова

овозможува автоматско подредување на јазлите според нивната фреквенција, така што алгоритмот секогаш може да ги избере двата јазли со најмала фреквенција за понатамошно спојување. Ова е основниот принцип врз кој се базира конструкцијата на *Huffman* дрвото.

На концептуално ниво, класата *Node* претставува фундаментална градбена единица на алгоритмот. Секој симбол започнува како посебен јазол, а потоа преку последователни спојувања се создава целосното *Huffman* дрво. Откако дрвото ќе биде изградено, неговата структура се користи за генерирање на бинарните кодови, при што секоја патека од коренот до лист претставува единствен коден збор за соодветниот симбол.

```
def build_tree(frequencies):
    heap = []
    for symbol, freq in frequencies.items():
        heapq.heappush(heap, Node(symbol, freq))
    while len(heap) > 1:
        left = heapq.heappop(heap)
        right = heapq.heappop(heap)
        merged = Node(None, left.freq + right.freq)
        merged.left = left
        merged.right = right
        heapq.heappush(heap, merged)
    return heap[0]
```

4.3. Функција *build_tree(frequencies)*

По пресметувањето на фреквенциите на симболите, следниот чекор во *Huffman* алгоритмот е изградба на бинарно дрво кое ќе послужи како основа за генерирање на кодните зборови.

Функцијата *build_tree* како аргумент прима структура од тип речник (dictionary) во која за секој симбол е наведена неговата фреквенција на појавување на текстот. Целта е од оваа информација да се изгради *Huffman* дрво кое ги минимизира пресечната должина на кодот и вкупниот број потребни битови.

На почетокот се создава празна приоритетна редица (*heap*). Во неа за секој симбол се креира посебен јазол од класата *Node*, при што секој јазол ја содржи фреквенцијата на соодветниот симбол. Приоритетната редица автоматски ги одржува јазлите подредени според нивната фреквенција, така што најмалите вредности секогаш се наоѓаат на врвот на структурата.

По иницијализацијата започнува главниот дел од алгоритмот. Се додека во редицата постојат повеќе од еден јазол, се земаат двата јазли со најмала фреквенција. Овие два јазли се спојуваат во нов внатрешен јазол чија фреквенција е еднаква на збирот од нивните фреквенции.

Математички, за новиот јазол важи:

$$f_{new} = f_{left} + f_{right}$$

каде:

- f_{left} е фреквенцијата на левиот јазол;
- f_{right} е фреквенцијата на десниот јазол.

Новиот јазол не претставува конкретен симбол и затоа неговиот атрибут *symbol* има вредност none. Неговите лево и десно поддрво се поставуваат на двата јазли кои биле избрани од приоритетната редица.

Откако ќе се создаде новиот јазол, тој повторно се внесува во редицата. На овој начин бројот на јазли постепено се намалува, а структурата на дрвото постепено расте.

Процесот продолжува се додека во редицата не остане само еден јазол. Последниот преостанат јазол претставува корен на целосното *Huffman* дрво и се враќа како резултат од функцијата.

Алгоритмот може да се замисли како последователно групирање на најретките симболи. Бидејќи симболите со мала фреквенција се наоѓаат подлабоко во

дрвото, тие добиваат подолги кодови. Токму ова својство овозможува *Huffman* кодирањето да биде оптимален метод за префиксно кодирање.

На крајот од извршувањето, функцијата враќа целосно изградено *Huffman* дрво кое подоцна се користи за генерирање на бинарните кодови и компресија на податоците.

```
generate_codes(node, code="", codes=None):
    if codes is None:
        codes = {}
    if node is None:
        return codes
    if node.symbol is not None:
        codes[node.symbol] = code
    generate_codes(node.left, code + "0", codes)
    generate_codes(node.right, code + "1", codes)
    return codes
```

4.4. функција *generate_codes()*

По успешното конструирање на *Huffman* дрвото, потребно е секој симбол да добие соодветен бинарен код. Оваа задача ја извршува функцијата *generate_codes()*, прикажана на слика 4.4., која преку рекурзивно поминување низ дрвото ги генерира кодните зборови за сите симболи.

Основната идеја на функцијата се базира на фактот дека секоја патека од коренот на дрвото до некој лист претставува единствен *Huffman* код. При движење кон левото поддрво се додава битот 0, а при движење кон десното поддрво се додава битот 1. Со последователно додавање на овие битови се формира бинарниот код на симболот.

Функцијата прима три параметри:

- *node* – тековниот јазол што се обработува;
- *code* – моментално формираните бинарен код;
- *codes* – речник во кој се складираат генерираните кодови.

На почетокот се проверува дали речникот за кодови постои. Доколку не е иницијализиран, се создава празен речник кој ќе ги содржи сите генерирани кодови.

Потоа следува проверка дали тековниот јазол постои. Доколку е достигнат крај на гранката и јазолот има вредност *none*, функцијата ја прекинува обработката на таа гранка и го враќа моменталниот речник со кодови.

Следниот важен чекор е проверката дали тековниот јазол претставува лист во дрвото. Листовите се единствените јазли кои содржат реални симболи од азбуката. Кога ќе се достигне таков јазол, моментално формируваниот бинарен код се доделува на соодветниот симбол и се зачувува во речникот.

По обработката на тековниот јазол, функцијата рекурзивно ги посетува неговите деца. При премин кон левата гранка се додава битот 0 на моменталниот код, додека при премин кон десната гранка се додава битот 1. На овој начин секоја патека низ дрвото создава различна комбинација од битови.

Забележливо е дека почестите симболи обично се наоѓаат поблиску до коренот и добиваат пократки кодови, додека поретките симболи се поставени подлабоко во дрвото и добиваат подолги кодови.

На крајот од извршувањето, функцијата враќа речник во кој секој симбол е поврзан со својот уникатен бинарен код. Овој речник подоцна се користи за кодирање на текстот и пресметка на степенот на компресија.

Математички гледано, функцијата ја претвора дрвовидната структура генерирана во претходната фаза во практичен коден речник, кој претставува конечен резултат на *Huffman* алгоритмот и директно се користи во процесот на компресија на податоците.


```

def show_huffman_tree(tree):
    G = nx.DiGraph()
    def add(node, parent=None, label=""):
        if node is None:
            return
        node_id = str(id(node))
        if node.symbol is None:
            node_label = f"{node.freq}"
        else:
            node_label = f"{node.symbol}\n({node.freq})"
        G.add_node(node_id, label=node_label)
        if parent:
            G.add_edge(parent, node_id, label=label)
            add(node.left, node_id, "0")
            add(node.right, node_id, "1")
    add(tree)
    pos = nx.kamada_kawai_layout(G)
    labels = nx.get_node_attributes(G, "label")
    edge_labels = nx.get_edge_attributes(G, "label")
    plt.figure(figsize=(10, 6))
    nx.draw(G, pos, labels=labels, with_labels=True, node_size=2000)
    nx.draw_networkx_edge_labels(
        G,
        pos,
        edge_labels=edge_labels
    )
    plt.title("Huffman Tree")
    plt.show()

```

4.5. функција `show_huffman_tree(tree)`

Функцијата `show_huffman_tree()`, прикажана на слика 4.5., има задача визуелно да го претстави *Huffman* дрвото генерирано од алгоритмот. Функцијата не учествува во процесот на кодирање, туку служи за графичка интерпретација на добиената структура.

Најпрво се формира графичка интерпретација на дрвото, при што секој јазол се претставува со неговата ознака и фреквенција. Потоа се воспоставуваат врските помеѓу родителските и детските јазли, при што се означуваат левите и десните гранки.

По формирањето на графот, автоматски се пресметува распоредот на јазлите и се генерира визуелен приказ на дрвото. Дополнително се прикажуваат и ознаките на гранките, што овозможува полесно следење на патеките кои ги формираат кодните зборови.

```
def plot_frequencies(freq):  
    symbols = list(freq.keys())  
    values = list(freq.values())  
    plt.figure(figsize=(8, 4))  
    plt.bar(symbols, values)  
    plt.title("Symbol Frequencies")  
    plt.show()
```

4.6. функција *plot_frequencies(freq)*

Функцијата *plot_frequencies()* има задача графички да ја прикаже распределбата на симболите во анализираниот текст. Нејзината улога е да обезбеди визуелен увид во зачестеноста на појавување на секој симбол. Функцијата како влезен параметар прима речник кој ги содржи симболите и нивните соодветни фреквенции. Овој речник претходно се генерира со анализа на внесениот текст, при што за секој симбол се брои колку пати се појавува. Од добиената структура се издвојуваат две посебни листи: една која ги содржи симболите и друга која ги содржи нивните фреквенции.

За визуелизација се користи библиотеката *Matplotlib*, која овозможува генерирање на различни видови графици. Во овој случај се креира столбест дијаграм (bar chart), при што секој столб претставува еден симбол во текстот, а неговата висина ја претставува фреквенцијата на појавување на тој симбол.

На хоризонталната оска се прикажуваат симболите, додека на вертикалната оска се прикажува бројот на нивните појавувања.

Ваквата анализа има особено значење во контекст на Теоријата на информации. Распределбата на фреквенциите директно влијае врз ентропијата на изворот и врз ефикасноста на алгоритмите за кодирање. Поради тоа, графикот претставува визуелна потврда на статистичката структура на податоците.

```

def create_huffman_tab(parent):
    def run_huffman():
        text = input_text.get("1.0", tk.END).strip()
        if not text:
            return
        freq = Counter(text)
        tree = build_tree(freq)
        codes = generate_codes(tree)
        encoded_text = "".join(codes[c] for c in text)
        show_huffman_tree(tree)
        plot_frequencies(freq)
        result = "HUFFMAN CODES\n\n"
        for symbol, code in sorted(codes.items()):
            if symbol == " ":
                symbol = "[SPACE]"
            elif symbol == "\n":
                symbol = "[NEWLINE]"
            result += f"{symbol} : {code}\n"
        result += "\n"
        result += f"Original length: {len(text) * 8} bits\n"
        result += f"Compressed length: {len(encoded_text)} bits\n"
        result += f"Compression ratio: {len(encoded_text)/(len(text)*8):.3f}\n"
        output_text.delete("1.0", tk.END)
        output_text.insert(tk.END, result)

```

4.7. функција `run_huffman()`

Функцијата `run_huffman()`, претставува централна функција на `huffman.py` модулот и ја координира работата на сите претходно дефинирани функции. Нејзина задача е да го преземе текстот од корисникот, да го обработи со *Huffman* алгоритмот и да ги прикаже добиените резултати.

На почетокот функцијата го чита внесениот текст и проверува дали е внесена валидна содржина. Доклоку текстот е празен, обработката се прекинува. Потоа се пресметуваат фреквенциите на сите симболи и врз основа на нив се конструира *Huffman* дрвото. Откако дрвото ќе биде изградено, се генерираат бинарните кодови за секој симбол и се врши кодирање на целата порака.

Дополнително, функцијата ги повикува функциите за визуелизација на *Huffman* дрвото и графичкиот приказ на фреквенциите на симболите.

Во завршниот дел се формира извештај кој ги содржи генерираните *Huffman* кодови, должината на оригиналната порака, должината на компресираната порака и односот на компресија. Резултатите се прикажуваат во излезниот прозорец на апликацијата.

4.3. Алгоритам на *Shannon – Fano* (*shannon_fano.py*)

Shannon_Fano модулот има за цел имплементација на алгоритмот за префикс кодирање познат како *Shannon_Fano* кодирање. Овој алгоритам, слично на *Huffman* кодирањето, се користи за компресија на податоци преку доделување на бинарни кодови со променлива должина на симболите, врз основа на нивната веројатност на појавување.

Основната идеја на *Shannon_Fano* алгоритмот е симболите да се подредат според нивната фреквенција, по што рекурзивно се делат на две групи со приближно еднаква вкупна веројатност. На симболите од првата група им се доделува префикс 0, додека на симболите од втората група им се доделува префикс 1. Овој процес се повторува рекурзивно за секоја група се додека секој симбол не добие единствен бинарен код.

```

def shannon_fano(symbols, codes=None):
    if codes is None:
        codes = {}
    if len(symbols) == 1:
        symbol, _ = symbols[0]
        if symbol not in codes:
            codes[symbol] = "0"
        return codes
    total = sum(freq for _, freq in symbols)
    running_sum = 0
    split_index = 0
    for i, (_, freq) in enumerate(symbols):
        running_sum += freq
        if running_sum >= total / 2:
            split_index = i
            break
    left = symbols[:split_index + 1]
    right = symbols[split_index + 1:]
    for symbol, _ in left:
        codes[symbol] = codes.get(symbol, "") + "0"
    for symbol, _ in right:
        codes[symbol] = codes.get(symbol, "") + "1"
    if left:
        shannon_fano(left, codes)
    if right:
        shannon_fano(right, codes)
    return codes

```

4.8. функција *shannon_fano()*

Функцијата *shannon_fano()*, прикажана на слика 4.8., ја реализира основната постапка на *Shannon – Fano* кодирањето. Нејзината задача е врз основа на фреквенциите на симболите да генерира префиксни бинарни кодови со променлива должина.

Алгоритмот започнува со сортирана листа на симболи според нивната фреквенција на појавување. Основната идеја е множеството симболи рекурзивно да се дели на две групи чија вкупна фреквенција е што е можно поблиска. На симболите од првата група им се додава битот 0, а на симболите од втората група

им се додава битот 1. Потоа истата постапка се повторува за секој од новодобиените групи се додека секоја група не содржи само еден симбол.

На почетокот од функцијата се проверува дали постои речник за складирање на кодовите. Доколку не е креиран, се иницијализира празен речник во кој постепено ќе се запишуваат кодните зборови.

Следниот чекор е проверка дали тековната листа содржи само еден симбол. Ова претставува базен случај на рекурзијата. Доколку е достигната ваква состојба, симболот веќе има формиран код и функцијата ја завршува обработката на таа гранка.

Главниот дел од алгоритмот се состои од пресметување на вкупната фреквенција на сите симболи во тековната група и пронаоѓање на точката на поделба. За таа цел постепено се собираат фреквенциите на симболите се додека нивниот збир не достигне приближно половина од вкупната фреквенција.

Математички, целта е да се најде поделба при која важи:

$$\sum_{i \in Left} f_i \approx \sum_{j \in Right} f_j$$

каде што:

- f_i се фреквенциите на симболите во левата група;
- f_j се фреквенциите на симболите во десната група.

Откако ќе се одреди точката на поделба, множеството симболи се разделува на две подгрупи. На сите симболи од левата група се додава битот 0, а на симболите од десната група битот 1. На овој начин постепено се формираат кодните зборови.

Потоа функцијата рекурзивно се повикува за двете новоформирани групи. Со секое ново ниво на рекурзија кодовите стануваат подолги и попрецизни, се додека секој симбол не добие уникатен бинарен код.

Карактеристично за *Shannon – Fano* алгоритмот е што должината на кодот зависи од фреквенцијата на симболот. Симболите со поголема веројатност најчесто се наоѓаат во помал број поделби и добиваат пократки кодови, додека поретките симболи учествуваат во повеќе поделби и добиваат подолги кодни зборови.

На крајот од извршувањето функцијата враќа речник во кој секој симбол е поврзан со својот *Shannon – Fano* код. Овој речник понатаму се користи за кодирање на текстот и пресметување на степенот на компресија.

```
def build_shannon_codes(text):  
    freq = Counter(text)  
    sorted_symbols = sorted(  
        freq.items(),  
        key=lambda x: x[1],  
        reverse=True  
    )  
    codes = shannon_fano(sorted_symbols)  
    return codes, freq
```

4.9. функција `build_shannon_codes()`

Функцијата `build_shannon_codes()`, претставува помошна функција чија задача е да ги подготви влезните податоци за *Shannon – Fano* алгоритмот и да го иницира процесот на генерирање на кодните зборови.

Како влезен параметар функцијата прима текстуална низа која треба да биде кодирана. Во првиот чекор се пресметува фреквенцијата на појавување на секој симбол во текстот. За таа цел се користи структура кој за секој симбол го евидентира бројот на неговите појавувања. Добиените фреквенции претставуваат статистички опис на информацискиот извор и служат како основа за понатамошното кодирање.

По пресметувањето на фреквенциите, симболите се подредуваат во опаѓачки редослед според нивната зачестеност. Овој чекор е неопходен бидејќи *Shannon – Fano* алгоритмот претпоставува симболите да бидат сортирани од најчест кон

најредок. Само на тој начин може правилно да се изврши рекурзивното делење на множеството на две подгрупи со приближно еднакви вкупни фреквенции.

Откако ќе се формира сортираната листа, таа се проследува до функцијата *run_shannon()*, која го извршува главниот алгоритам и генерира бинарни кодови за сите симболи. Резултатот е речник во кој секој симбол е поврзан со својот коден збор.

На крајот функцијата враќа две вредности: генерираните *Shannon – Fano* кодови и пресметаните фреквенции на симболите. Враќањето на фреквенциите овозможува нивно понатамошно користење во други делови од апликацијата, како што се статистичките анализи и графичките визуелизации.

Од аспект на архитектурата на апликацијата, оваа функција претставува посредник помеѓу корисничкиот внес и *Shannon – Fano* алгоритмот. Таа ги организира податоците во форма погодна за обработка и обезбедува правилно повикување на алгоритмот за кодирање.

```
def plot_frequencies(freq):  
    symbols = list(freq.keys())  
    values = list(freq.values())  
    plt.figure(figsize=(8,4))  
    plt.bar(symbols, values)  
    plt.title("Symbol Frequencies")  
    plt.show()
```

4.10. функција *plot_frequencies()*

Функцијата *plot_frequencies()*, прикажана погоре на сликата 4.10., има задача графички да ја претстави распределбата на фреквенциите на симболите во анализираниот текст. Како влезен параметар прима речник кој ги содржи симболите и бројот на нивните појавувања.

Од добиените податоци се издвојуваат симболите и нивните фреквенции, по што со помош на библиотеката *Matplotlib* се генерира столбест дијаграм. На

хоризонталната оска се прикажуваат симболите, додека на вертикалната оска се прикажува нивната фреквенција на појавување.

Бидејќи идентична визуелизација се користи и во *huffman.py* модулот, улогата на оваа функција е првенствено информативна. Таа му овозможува на корисникот брзо да ја согледа статистичката распределба на симболите врз која се базира *Shannon – Fano* алгоритмот и да ги спореди зачестеностите на поединечните симболи пред процесот на кодирање.

```
def create_shannon_tab(parent):
    def run_shannon():
        text = input_text.get("1.0",tk.END).strip()
        if not text:
            return
        frequencies = Counter(text)
        sorted_symbols = sorted(
            frequencies.items(),
            key=lambda x: x[1],
            reverse=True
        )
        codes = shannon_fano(sorted_symbols)
        plot_frequencies(frequencies)
        encoded_text = ""
        for char in text:
            encoded_text += codes[char]
        result = "SHANNON-FANO CODES\n\n"
        for symbol, code in codes.items():
            if symbol == " ":
                display_symbol = "[SPACE]"
            else:
                display_symbol = symbol
            result += (f"{display_symbol} : {code}\n")
        result += "\n"
        result += (f"Original length: {len(text) * 8} bits\n")
        result += (f"Compressed length: {len(encoded_text)} bits\n")
        ratio = (len(encoded_text)/ (len(text) * 8))
        result += (f"Compression ratio: {ratio:.3f}\n")
        output_text.delete("1.0",tk.END)
        output_text.insert(tk.END,result)
```

4.11. функција *run_shannon()*

Функцијата `run_shannon()` е еквивалент на функцијата `run_huffman()` од `huffman.py` модулот. Функцијата претставува главна контролна функција на `shannon_fano.py` модулот. Нејзината задача е да го преземе текстот внесен од корисникот, да го обработи со *Shannon – Fano* алгоритмот и да ги прикаже добиените резултати.

Во првиот чекор се чита внесениот текст и се проверува дали е внесена валидна содржина. Доколку текстот е празен, обработката се прекинува. Потоа се пресметуваат фреквенциите на сите симболи и тие се подредуваат во опаѓачки редослед според нивната зачестеност, што претставува неопходен предуслов за правилно функционирање на *Shannon – Fano* алгоритмот.

Врз основа на добиената распределба на фреквенции се повикува функцијата `build_shannon_codes()`, која ги подготвува податоците потребни за *Shannon – Fano* кодирањето. Дополнително се повикува и функцијата `plot_frequencies()`, со што корисникот добива графички приказ на статистичката распределба на симболите во текстот.

По генерирањето на кодовите се врши кодирање на целата порака, при што секој симбол се заменува со соодветниот *Shannon – Fano* код. Добиената бинарна низа се користи за пресметување на должината на компресираната порака и на односот на компресија. Во завршниот дел од функцијата се формира извештај кој ги содржи генерираните кодови за сите симболи, должината на оригиналната порака изразена во битови, должината на компресираната порака и постигнатиот коефициент на компресија. Резултатите се прикажуваат во излезниот прозорец на апликацијата.

4.4. Модул за анализа на кодови (`code_checker.py`)

Модулот за анализа на кодови е наменет за проверка и евалуација на својствата на кодовите кои се користат при претставување и пренос на информации. Неговата основна цел е да утврди дали даден код ги задоволува условите

потребни за сигурно и еднозначно декодирање, како и да обезбеди дополнителни информации за неговата структура и ефикасност.

Во Теоријата на информации не е доволно само да се генерираат кодни зборови. Подеднакво важно е да се провери дали тие кодови можат еднозначно да се декодираат и дали ги исполнуваат математичките услови кои гарантираат правилна интерпретација на податоците. Поради тоа, овој модул претставува важна алатка за анализа на квалитетот на кодовите добиени со различни алгоритми.

```
def is_prefix_free(codes):
    codewords = list(codes.values())
    for i in range(len(codewords)):
        for j in range(len(codewords)):
            if i != j:
                if codewords[j].startswith(codewords[i]):
                    return False
    return True
```

4.12. Функција *is_prefix_free()*

Функцијата *is_prefix_free()*, има задача да провери дали дадено множество кодни зборови претставува префиксен код. Во Теоријата на информации, кодот е префиксен доколку ниту еден коден збор не претставува почетен дел (префикс) од друг коден збор. Ова својство овозможува еднозначно декодирање на пораките без потреба од дополнителни разделувачи помеѓу кодните зборови.

Како влезен параметар функцијата прима речник со кодови, при што најпрво се издвојуваат само кодните зборови. Потоа се врши споредување на секој коден збор со сите останати кодни зборови во множеството.

За секој пар кодови се проверува дали едниот код започнува со другиот. Доколку се пронајде таков случај, тоа значи дека еден коден збор е префикс на друг и кодот не го задоволува условот за префиксност. Во таа ситуација функцијата враќа вредност *false*.

Доколку по завршувањето на сите споредби не се пронајде ниту еден таков пар, функцијата враќа вредност `true`, што значи дека кодот е префиксен и може еднозначно да се декодира.

```
def kraft_inequality(codes):  
    total = 0  
    for code in codes.values():  
        total += 2 ** (-len(code))  
    return total
```

4.13. Функција `kraft_inequality()`

Функцијата `kraft_inequality()`, има задача да ја пресмета вредноста од левата страна од Kraft –овото неравенство а дадено множество кодни зборови. Kraft –овото неравенство претставува еден од основните резултати во Теоријата на информации и се користи за проверка дали постои префиксен код со зададените должини на кодните зборови.

За бинарни кодови, Kraft –овото неравенство е дефинирано со изразот:

$$\sum_{i=1}^n 2^{-l_i} \leq 1$$

каде што:

- l_i е должината на i -тиот коден збор;
- n е бројот на кодни зборови.

Функцијата како влезен параметар прима речник кој ги содржи кодните зборови. На почетокот се иницијализира променливата `total`, која ќе ја содржи сумата од сите членови на неравенството.

Потоа функцијата ги разгледува сите кодни зборови еден по еден. За секој код се пресметува вредноста 2^{-l_i} , каде што должината на кодниот збор се добива со функцијата `len()`. Добиената вредност се додава на вкупната сума.

По обработката на сите кодни зборови, функцијата ја враќа конечната вредност на сумата. Добиениот резултат потоа може да се спореди со бројот 1 за да се провери дали е исполнето Kraft –овото неравенство.

Во рамките на апликацијата оваа функција служи како дополнителна алатка за анализа на кодови. Таа не проверува директно дали кодот е префиксен, туку претставува теоретска величина која овозможува проценка дали должините на кодните зборови се во согласност со условите дефинирани од Kraft –овото неравенство.

```
def create_code_analysis_tab(parent):
    def analyze():
        raw = input_text.get("1.0", tk.END).strip().splitlines()
        codes = {}
        for line in raw:
            parts = line.split()
            if len(parts) == 2:
                symbol, code = parts
                codes[symbol] = code
        if not codes:
            output_text.delete("1.0", tk.END)
            output_text.insert(tk.END, "No valid input")
            return
        prefix = is_prefix_free(codes)
        kraft_sum = kraft_inequality(codes)
        result = "CODE ANALYSIS\n\n"
        result += "Prefix-free: " + str(prefix) + "\n\n"
        result += f"Kraft sum = {kraft_sum:.4f}\n"
        if kraft_sum <= 1:
            result += "Kraft condition: VALID ( $\leq 1$ )\n"
        else:
            result += "Kraft condition: INVALID ( $> 1$ )\n"
        result += "\nCodes:\n"
        for s, c in codes.items():
            if s == " ":
                s = "[SPACE]"
            result += f"{s} -> {c}\n"
        output_text.delete("1.0", tk.END)
        output_text.insert(tk.END, result)
```

4.14. Функција *analyze()*

Функцијата *analyze()*, претставува главна функција на модулот за анализа на кодови. Нејзина задача е да ги преземе кодните зборови внесени од корисникот и да ги проследи до функциите за анализа кои се имплементирани во модулот.

Најпрво од внесените податоци се формира речник кој ги содржи симболите и нивните кодни зборови. Доколку не постојат валидни податоци, функцијата ја прекинува обработката и прикажува соодветна порака.

Потоа се повикуваат функциите за проверка на префиксноста и пресметка на Kraft –ова сума. Добиените резултати се собираат во единствен извештај кој содржи информации за својствата на внесениот код.

Во завршниот дел извештајот се прикажува во излезниот прозорец на апликацијата заедно со листа на сите внесени кодни зборови.

Функцијата не имплементира нов алгоритам, туку служи како координатор кој ги обединува резултатите од претходно дефинираните анализи и ги претставува на корисникот во прегледна форма.

4.5. Модул за анализа на комуникациски канал (*channel.py*)

Модулот за анализа на комуникациски канал е наменет за проучување на преносот на информации помеѓу изворот и приемникот преку комуникациски канал. Неговата основна цел е да овозможи пресметка и анализа на клучните информациски мерки кои ја опишуваат количината на успешно пренесена информација, степенот на неизвесност и влијанието на каналот врз преносот на податоците.

Во Теоријата на информации, комуникацискиот канал претставува систем преку кој пораките се пренесуваат од изворот до дестинацијата. Во реални услови, преносот често е под влијание на шум и различни видови нарушувања, поради што приемниот сигнал не секогаш е идентичен со испратениот.

Во рамките на овој модул корисникот внесува веројатносни распределби кои го опишуваат однесувањето на каналот. Врз основа на внесените податоци,

апликацијата пресметува различни информациски мерки кои овозможуваат квантитативна анализа на преносот на информации.

```
def entropy(dist):
    h = 0
    for p in dist:
        if p > 0:
            h -= p * math.log2(p)
    return h
```

4.15. Функција *entropy()*

```
def create_channel_tab(parent):
    def calculate():
        try:
            px = list(map(float, px_entry.get().split()))
            matrix_raw = matrix_text.get("1.0", tk.END).strip().splitlines()
            p_y_given_x = []
            for row in matrix_raw:
                p_y_given_x.append(list(map(float, row.split())))
            py = [0 for _ in range(len(p_y_given_x[0]))]
            for i in range(len(px)):
                for j in range(len(py)):
                    py[j] += px[i] * p_y_given_x[i][j]
            hx = entropy(px)
            hy = entropy(py)
            hyx = 0
            for i in range(len(px)):
                for j in range(len(py)):
                    pxy = px[i] * p_y_given_x[i][j]
                    if pxy > 0:
                        hyx -= pxy * math.log2(p_y_given_x[i][j])
            ixy = hy - hyx
            result = "CHANNEL ANALYSIS\n\n"
            result += f"H(X) = {hx:.4f}\n"
            result += f"H(Y) = {hy:.4f}\n"
            result += f"H(Y|X) = {hyx:.4f}\n"
            result += f"I(X;Y) = {ixy:.4f}\n\n"
            result += f"P(Y) = {py}\n"
            output_text.delete("1.0", tk.END)
            output_text.insert(tk.END, result)
        except Exception as e:
            output_text.delete("1.0", tk.END)
            output_text.insert(tk.END, str(e))
```

4.16. Функција *calculate()*

Функцијата *calculate()*, прикажана на сликата погоре, претставува главна функција на модулот за анализа на комуникациски канал. Нејзината задача е врз основа на распределбата на влезните симболи и матрицата на условни веројатности да ги пресмета основните информациски параметри на каналот.

На почетокот функцијата ги чита веројатностите на влезниот извор $P(X)$ и матрицата на условни веројатности $P(Y|X)$ внесени од корисникот. Овие податоци го опишуваат однесувањето на комуникацискиот канал.

Следниот чекор е пресметување на распределбата на излезниот сигнал $P(Y)$. За секој излезен симбол се собира придонесот од сите можни влезни симболи според релацијата:

$$P(y_j) = \sum_i P(x_i) P(y_j|x_i)$$

Со добиената распределба се пресметува ентропијата на влезниот извор $H(X)$ и ентропијата на излезот $H(Y)$ со помош на претходно дефинираната функција *entropy()*.

Потоа се пресметува условната ентропија $H(Y|X)$, која ја претставува просечната неизвесност на излезот кога влезниот симбол е познат. За оваа величина се користи формулата:

$$H(Y|X) = - \sum_i \sum_j P(x_i) P(y_j|x_i) \log_2 P(y_j|x_i)$$

Откако ќе се добие условната ентропија, се пресметува заемната информација помеѓу влезот и излезот на каналот според изразот:

$$I(X, Y) = H(Y) - H(Y|X)$$

Заемната информација ја покажува количината на информација која излезот ја содржи за влезот и претставува една од најважните мерки за квалитетот на комуникацискиот канал.

Во завршниот дел функцијата ги прикажува пресметаните вредности на ентропијата на влезот, ентропијата на излезот, условната ентропија и заемната информација, како и добиената распределба на излезните симболи. Така, корисникот добива комплетна анализа на информациските карактеристики на разгледуваниот канал.

```
def show_channel(px, p_y_given_x):
    G = nx.DiGraph()
    for i in range(len(px)):
        for j in range(len(p_y_given_x[0])):
            G.add_edge(f"X{i}", f"Y{j}", weight=p_y_given_x[i][j])
    pos = {}
    for i in range(len(px)):
        pos[f"X{i}"] = (0, -i)
    for j in range(len(p_y_given_x[0])):
        pos[f"Y{j}"] = (2, -j)
    labels = nx.get_edge_attributes(G, "weight")
    plt.figure(figsize=(8,5))
    nx.draw(G, pos, with_labels=True, node_size=2500)
    nx.draw_networkx_edge_labels(G, pos, edge_labels=labels)
    plt.title("Communication Channel")
    plt.show()
```

4.17. Функција `show_channel()`

Функцијата `show_channel()` служи за графичко претставување на комуникацискиот канал дефиниран преку распределбата на влезните симболи и матрицата на условни веројатности.

На почетокот се креира насочен граф во кој јазлите од типот X_i ги претставуваат влезните симболи, а јазлите од типот Y_j ги претставуваат излезните симболи на каналот. Потоа за секоја вредност од матрицата $P(Y|X)$ се формира врска помеѓу соодветниот влезен и излезен симбол.

Вредноста на условната веројатност се зачувува како ознака на гранката, со што секоја врска ја претставува веројатноста одреден влезен симбол да произведе конкретен излезен симбол.

По формирањето на графот се пресметува распоредот на јазлите и се исцртува целосната структура на каналот. Дополнително се прикажуваат и ознаките на гранките, што овозможува визуелно следење на сите можни преноси низ каналот.

4.6. Модул за компресија на податоци ([compression.py](#))

Модулот за компресија на податоци претставува практична примена на концептите од Теорија на информации и има за цел да покаже како статистичките својства на податоците можат да се искористат за намалување на потребниот простор за нивно складирање и пренос. Преку овој модул корисникот може директно да ја согледа врската помеѓу ентропијата на изворот, методите за кодирање и степенот на постигнатата компресија.

```

def create_compression_tab(parent):
    def run():
        text = input_text.get("1.0", tk.END).strip()
        if not text:
            return
        freq = Counter(text)
        tree = build_tree(freq)
        codes = generate_codes(tree)
        encoded_text = "".join(codes[ch] for ch in text)
        original_bits = len(text) * 8
        compressed_bits = len(encoded_text)
        compression_ratio = compressed_bits / original_bits
        probabilities = [f / len(text) for f in freq.values()]
        entropy = 0
        for p in probabilities:
            if p > 0:
                entropy -= p * math.log2(p)
        average_length = 0
        for symbol, frequency in freq.items():
            probability = frequency / len(text)
            average_length += probability * len(codes[symbol])
        efficiency = entropy / average_length
        result = "COMPRESSION ANALYSIS\n\n"
        result += f"Original size: {original_bits} bits\n"
        result += f"Compressed size: {compressed_bits} bits\n"
        result += f"Compression ratio: {compression_ratio:.4f}\n\n"
        result += f"Entropy H(X): {entropy:.4f} bits/symbol\n"
        result += f"Average code length L: {average_length:.4f} bits/symbol\n"
        result += f"Efficiency  $\eta$ : {efficiency:.4f}\n\n"
        result += "Generated Huffman Codes:\n"
        for symbol, code in sorted(codes.items()):
            if symbol == " ":
                display_symbol = "[SPACE]"
            elif symbol == "\n":
                display_symbol = "[NEWLINE]"
            else:
                display_symbol = symbol
            result += f"{display_symbol} -> {code}\n"
        output_text.delete("1.0", tk.END)
        output_text.insert(tk.END, result)

```

4.18. Функција `run()`

Функцијата `run()`, прикажана на слика 4.18., претставува централен дел од модулот за анализа на компресија. Нејзина задача е да ја организира комуникацијата помеѓу корисничкиот интерфејс и алгоритмот за *Huffman*

кодирање, како и да ги пресмета основните параметри со кои се оценува ефикасноста на компресијата.

Во рамките на функцијата е дефинирана внатрешна функција *run()*, која се активира по избор на соодветното копче од графичкиот интерфејс. На почетокот се презема текстот внесен од корисникот и се отстрануваат евентуалните празни места на почетокот и крајот на пораката. Доколку не е внесен текст, функцијата го прекинува извршувањето.

Следниот чекор е пресметување на фреквенцијата на појавување на секој симбол во текстот. Врз основа на добиените фреквенции се конструира Huffman дрво и се генерираат соодветните кодни зборови. Потоа секој симбол од оригиналната порака се заменува со кодот, при што се добива компресираната бинарна низа.

За да се оцени степенот на компресија, се пресметуваат должината на оригиналната и должината на компресираната порака. Оригиналната должина се определува под претпоставка дека секој знак е претставен со 8 бита, додека компресираната должина е еднаква на бројот на битови во генерираната Huffman низа. Од овие две величини се добива компресираниот однос:

$$CR = \frac{N_c}{N_0}$$

Каде што N_c ја претставува должината на компресираната порака, а N_0 должината на оригиналната порака.

Покрај степенот на компресија, функцијата ја пресметува и ентропијата на информацискиот извор. Најпрво се определуваат веројатностите на појавување на сите симболи, а потоа се применува Shannon –овата формула за ентропија.

Потоа се пресметува просечната должина на кодот, која ја претставува очекуваната должина на кодниот збор а еден случајно избран симбол.

На крај се определува ефикасноста на кодирањето, која покажува колку просечната должина на кодот е блиска до теоретската граница зададена со ентропијата.

Вредностите добиени од анализата се прикажуваат во излезниот прозорец , заедно со генерираните кодови за секој симбол

Основната идеја на компресијата е информацијата да се претстави со помал број битови, без да се изгуби нејзината содржина. Во контекст на овој проект, компресијата се реализира преку алгоритмот на *Huffman* ,кој генерира кодови со променлива должина врз основа на зачестеноста на симболите во текстот. Симболите кои се појавуваат почесто добиваат пократки кодови, додека поретките симболи добиваат подолги кодови, со што се намалува просечната должина на кодираниот запис.

4.7. Модул за споредба ([comparison.py](#))

Модулот *comparison.py* овозможува споредба помеѓу различни методи на кодирање, што пак овозможува анализа на нивната ефикасност за конкретниот текст.

```

def compare_methods(text):
    freq_h = Counter(text)
    tree = build_tree(freq_h)
    huffman_codes = generate_codes(tree)
    huffman_encoded = ""
    for c in text:
        huffman_encoded += huffman_codes[c]
    freq_s = Counter(text)
    sorted_symbols = sorted(freq_s.items(),key=lambda x: x[1],reverse=True)
    shannon_codes = shannon_fano(sorted_symbols)
    shannon_encoded = ""
    for c in text:
        shannon_encoded += shannon_codes[c]
    original_bits = len(text) * 8
    huffman_bits = len(huffman_encoded)
    shannon_bits = len(shannon_encoded)
    return {
        "original": original_bits,
        "huffman": huffman_bits,
        "shannon": shannon_bits
    }

```

4.19. Функција `compare_methods()`

Функцијата `compare_methods()` врши споредба и нејзина задача е да ги спореди резултатите добиени со *Huffman* и *Shannon – Fano* кодирањето за ист влезен текст. За разлика од претходните модули , овде не се анализира само еден алгоритам , туку се врши паралелна примена на двата метода со цел да се споредат нивните компресиски способности.

На почетокот функцијата ја пресметува распределбата на фреквенциите на симболите во текстот и врз основа на неа го конструира *Huffman* дрвото. Потоа се генерираат кодовите и се формира компресираната порака преку замена на секој симбол со неговиот соодветен бинарен код. Како резултат се добива кодирана низа чија должина претставува број на битови потребни за претставување на пораката со *Huffman* кодирањето.

Во следната фаза се врши истата постапка со *Shannon – Fano* алгоритмот. Симболите најпрво се подредуваат според нивната фреквенција во опаѓачки редослед, бидејќи ова е предуслов за правилно функционирање на *Shannon –*

Fano постапката. Потоа се генерираат кодните зборови и се формира компресираната порака со замена на секој симбол со неговиот *Shannon – Fano* код.

Откако ќе се добијат двете компресирани претставувања, функцијата ја пресметува должината на оригиналната порака под претпоставка дека секој симбол е претставен со 8 бита. Потоа се определуваат должините на кодираните пораки, односно вкупниот број битови потребни за нивно претставување.

На крај функцијата враќа речник кој содржи три клучни вредности:

- Должината на оригиналната порака;
- Должината на *Huffman* кодираната порака;
- Должината на *Shannon – Fano* кодираната порака.

Со имплементација на овој модул се потврдува една од основните идеи на Теоријата на информации – дека степенот на компресија е тесно поврзан со ентропијата на изворот и дека оптималното кодирање претставува клучен механизам за обработка на податоците во современите информациски и комуникациски системи.

```
def plot_result(result):  
    methods = ["Huffman", "Shannon-Fano"]  
    values = [result["huffman"], result["shannon"]]  
    plt.figure(figsize=(6,4))  
    plt.bar(methods, values)  
    plt.title("Huffman vs Shannon-Fano (Compression in bits)")  
    plt.ylabel("Bits")  
    plt.show()
```

4.20. Функција *plot_result()*

Функцијата *plot_result()*, прикажана на слика ,служи за графичка визуелизација на резултатите добиени од споредбата меѓу двата алгоритма. Како влезен аргумент функцијата прима речник кој ги содржи должините на компресираните пораки

добие ни со двата алгоритми. Од овој речник се издвојуваат вредностите што го претставуваат бројот на битови потребни за кодирањето.

Со ова се формира столбест дијаграм (bar chart) во кој на хоризонталната оска се прикажани имињата на двата алгоритми, додека на вертикалната оска е претставен бројот на битови потребни за кодирање на пораката. Висината на секој столб ја претставува должината на компресираната порака добиена со соодветниот алгоритам.

```
def create_comparison_tab(parent):
    def run():
        text = input_text.get("1.0", tk.END).strip()
        if not text:
            return
        result = compare_methods(text)
        output_text.delete("1.0", tk.END)
        output_text.insert(tk.END, "COMPARISON RESULT\n\n")
        output_text.insert(tk.END, f"Original: {result['original']} bits\n")
        output_text.insert(tk.END, f"Huffman: {result['huffman']} bits\n")
        output_text.insert(tk.END, f"Shannon-Fano: {result['shannon']} bits\n\n")
        if result["huffman"] < result["shannon"]:
            better = "Huffman"
        elif result["shannon"] < result["huffman"]:
            better = "Shannon-Fano"
        else:
            better = "Equal Performance"
        output_text.insert(tk.END, f"Better: {better}\n")
        plot_result(result)
```

4.21. Функција `create_comparison_tab()`

5. Тестирање и анализа на резултатите (Case Study)

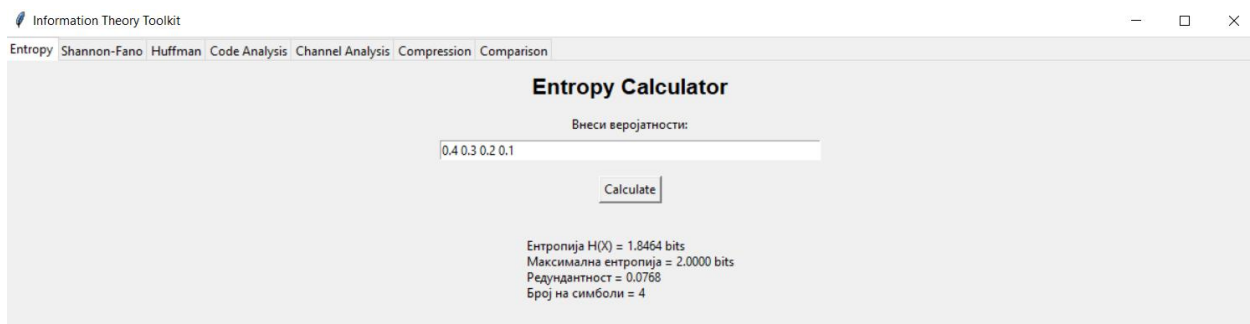
По имплементацијата на сите модули, извршено е тестирање на апликацијата со цел да се провери нејзината функционалност и практична примена на концептите од Теоријата на информации. За таа цел е избран примерен текст кој се користи како влезен податок во повеќе модули на системот.

Во текот на тестирањето се анализираат основните информациски параметри на изворот, се генерираат *Huffman* и *Shannon – Fano* кодови, се проверуваат својствата на добиените кодови, се испитуваат карактеристиките на комуникациски канал, како и можностите за компресија на податоците. Дополнително се врши споредба помеѓу двата алгоритми за кодирање со цел да се утврди нивната ефикасност при обработка на ист информациски извор.

Како текст пример е избрана пораката: *HELLO INFORMATION THEORY*

Изборот на оваа порака овозможува појава на повеќе различни симболи со различни фреквенции, што претставува соодветна основа за демонстрација на методите за кодирање и компресија.

5.1. Тест анализа на ентропија



The screenshot shows a web application titled "Information Theory Toolkit" with a navigation bar containing "Entropy", "Shannon-Fano", "Huffman", "Code Analysis", "Channel Analysis", "Compression", and "Comparison". The "Entropy" tab is active. The main section is titled "Entropy Calculator". It has a label "Внеси веројатности:" followed by a text input field containing "0.4 0.3 0.2 0.1". Below the input field is a "Calculate" button. The results displayed are: "Ентропија $H(X) = 1.8464$ bits", "Максимална ентропија = 2.0000 bits", "Редундантност = 0.0768", and "Број на симболи = 4".

5.1. Приказ на формата за внес на веројатности за пресметка на ентропија

На сликата 5.1., е прикажано извршувањето на модулот за пресметка на ентропија. Во полето се внесуваат веројатностите на појавување на симболите од информацискиот извор, по што со клик на копчето Calculate се пресметуваат ентропијата, максималната ентропија, редундантноста и бројот на симболи во азбуката. Добиените резултати овозможуваат проценка на количината на информација содржана во изворот.

За демонстрација на работата на модулот за ентропија беше избран информациски извор составен од четири симболи со распределба на веројатности:

$$P(X) = \{0.4, 0.3, 0.2, 0.1\}$$

По внесување на веројатностите во апликацијата се добиени следните резултати:

- Ентропија = 1.8464 бита
- Максимална ентропија = 2 бита
- Редундантност = 0.0768
- Број на симболи = 4

Ентропијата од 1.8464 бита ја претставува просечната количина на информација што ја носи еден симбол од изворот. Бидејќи вредноста е блиска до максималната ентропија од 2 бита, може да се заклучи дека изворот има релативно висока неизвесност и содржи значителна количина на информација.

За извор со четири симболи максималната ентропија се добива кога сите симболи се еднакво веројатни:

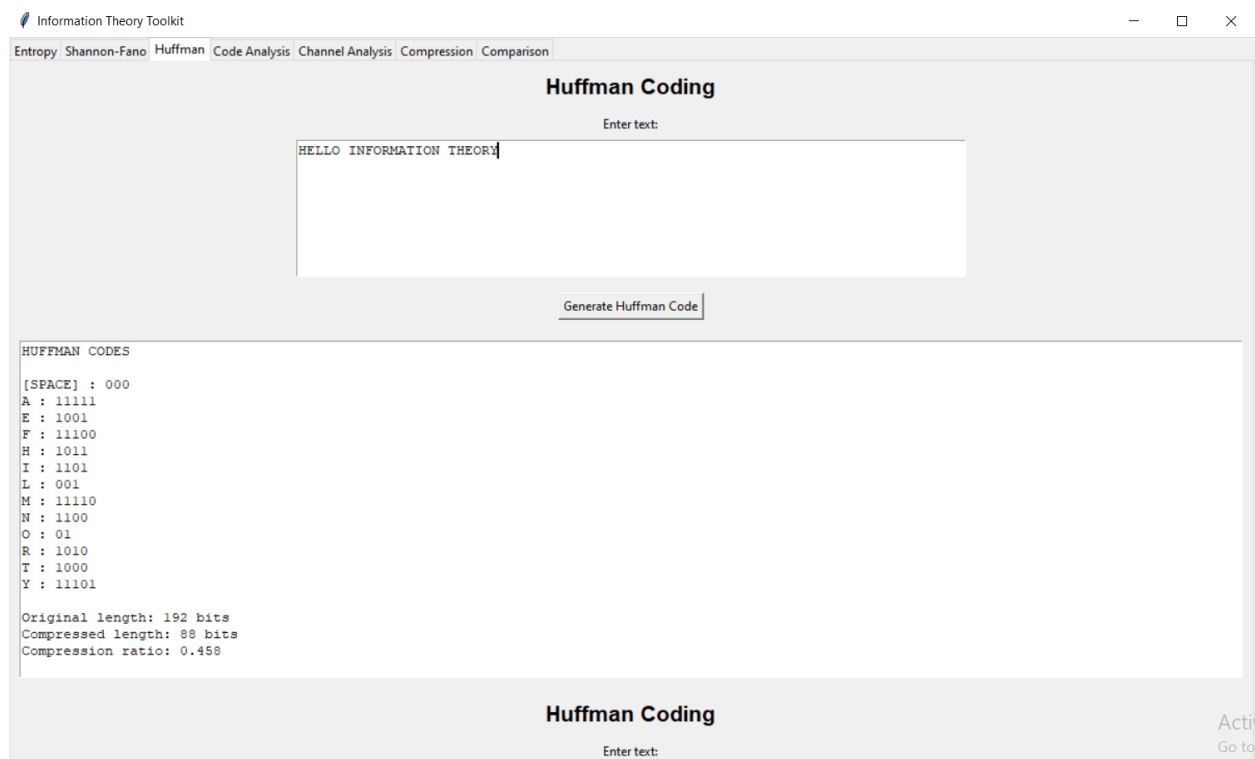
$$H_{max} = \log_2(4) = 2 \text{ bits}$$

Во разгледуваниот пример веројатностите не се целосно рамномерни, поради што ентропијата е нешто помала од теоретскиот максимум.

Редундантноста изнесува 0.0768, односно приближно 7.68%. ова значи дека во изворот постои релативно мала количина предвидливост и дека најголемиот дел од симболите придонесуваат со нова информација. Ниската редундантност укажува дека изворот е ефикасен од аспект на информациската содржина и дека не содржи повторувања.

Од добиените резултати може да се заклучи дека анализираниот извор е близок до случајно распределен извор со висока информативност. Поради релативно малата редундантност, можностите за дополнителна компресија се ограничени во споредба со извори кои содржат поголема предвидливост и поголем број повторувања на симболите.

5.2. Анализа на резултатите од *Huffman* кодирањето

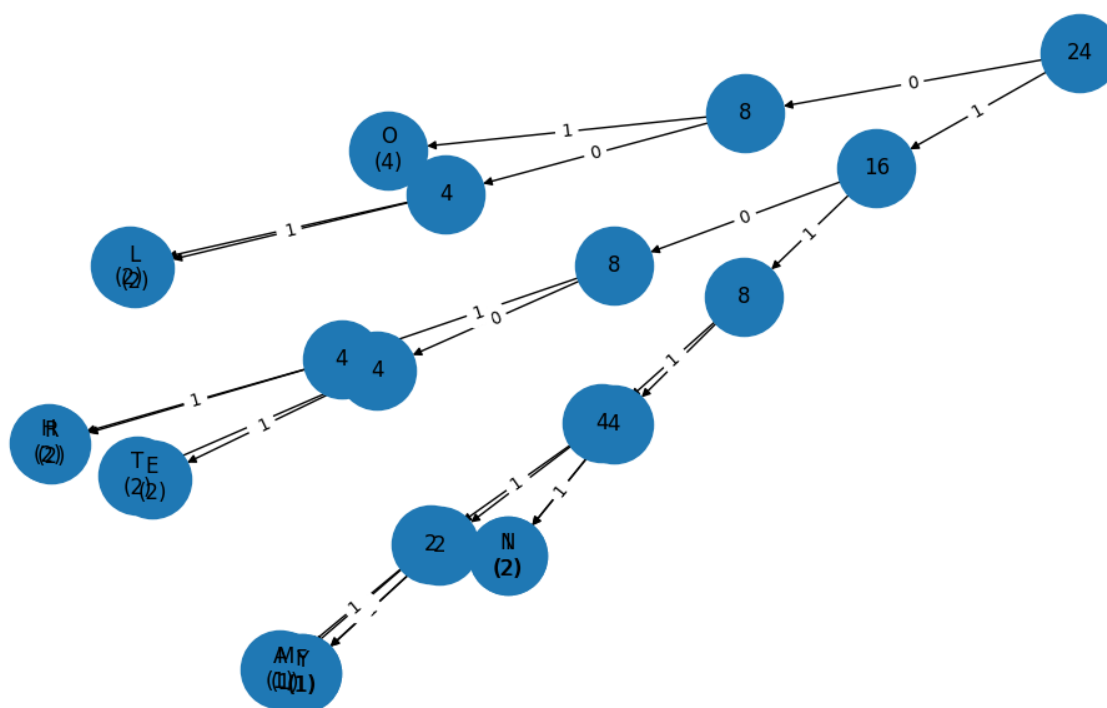


5.2. Приказ на формата за внес на пораката која ќе биде кодирана со *Huffman* алгоритмот

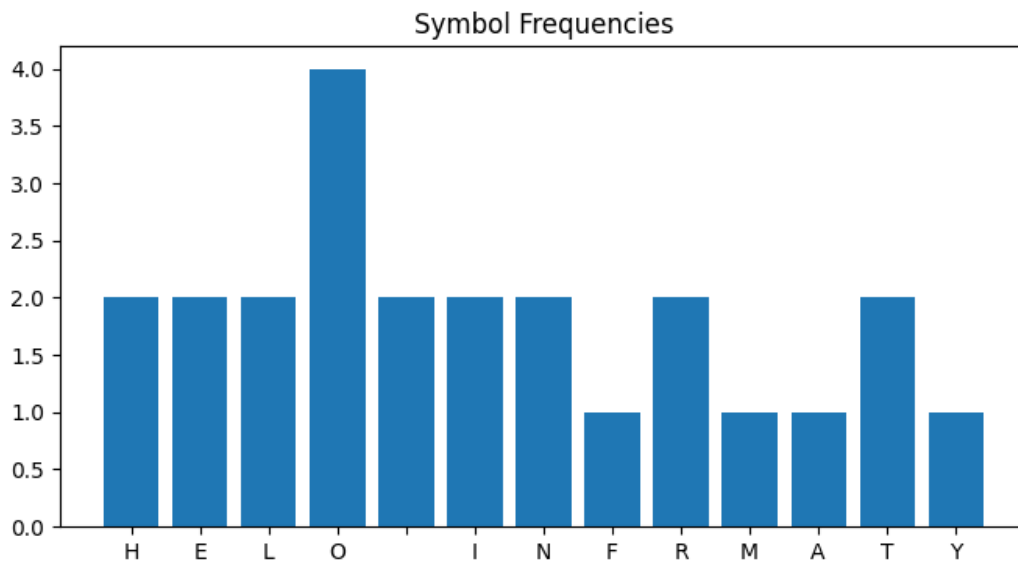
По внесување на текст пораката во *Huffman* модулот, и клик на копчето *Generate Huffman Code* апликацијата ја пресметува фреквенцијата на појавување на секој симбол, го конструира *Huffman* дрвото и генерира оптимални префиксни кодови. Врз основа на добиените кодови се формира компресирана бинарна претстава на пораката и се пресметуваат параметрите поврзани со компресијата.

На сликата 5.2., е прикажан корисничкиот интерфејс на *Huffman* модулот по извршената обработка на текст пораката. Во резултатите може да се забележи дека различните симболи добиваат кодови со различна должина, во зависност од нивната фреквенција на појавување. Симболите кои се појавуваат почесто добиваат пократки кодни зборови, додека поретките симболи добиваат подолги кодови.

Дополнително, апликацијата генерира и визуелен приказ на Huffman дрвото, преку кој може јасно да се следи процесот на формирање на кодните зборови. Секоја патека од коренот до лист претставува единствен бинарен код, при што преминот кон левото поддрво одговара на битот 0, а преминот кон десното поддрво на битот 1.



Покрај дрвото, се прикажува и график на фреквенции на симболите. Овој приказ овозможува визуелна потврда дека распределбата на должините на кодовите е поврзана со распределбата на фреквенциите во изворот. Симболите со поголема фреквенција се позиционирани поблиску до коренот на дрвото и добиваат пократки кодови, и обратно.



5.4. График на фреквенциите на симболите (*Huffman*)

Од добиените резултати може да се заклучи дека овој алгоритам успешно ја искористува статистичката структура на информацискиот извор за да ја намали должината на пораката.

За тест пораката *HELLO INFORMATION THEORY* апликацијата ги доби следните резултати:

- Оригинална должина на пораката: 192 бита
- Компресирана должина на пораката: 88 бита
- Компресиски однос: 0.458

Оригиналната должина е пресметана под претпоставка дека секој знак е претставен со 8 бита. Бидејќи пораката содржи 24 знаци, нејзината должина изнесува: $24 \cdot 8 = 192$

По примената на *Huffman* кодирањето, пораката е претставена со само 88 бита. Тоа значи дека бројот на потребни битови е значително намален без губење на информација.

Компресискиот однос е еднаков на :

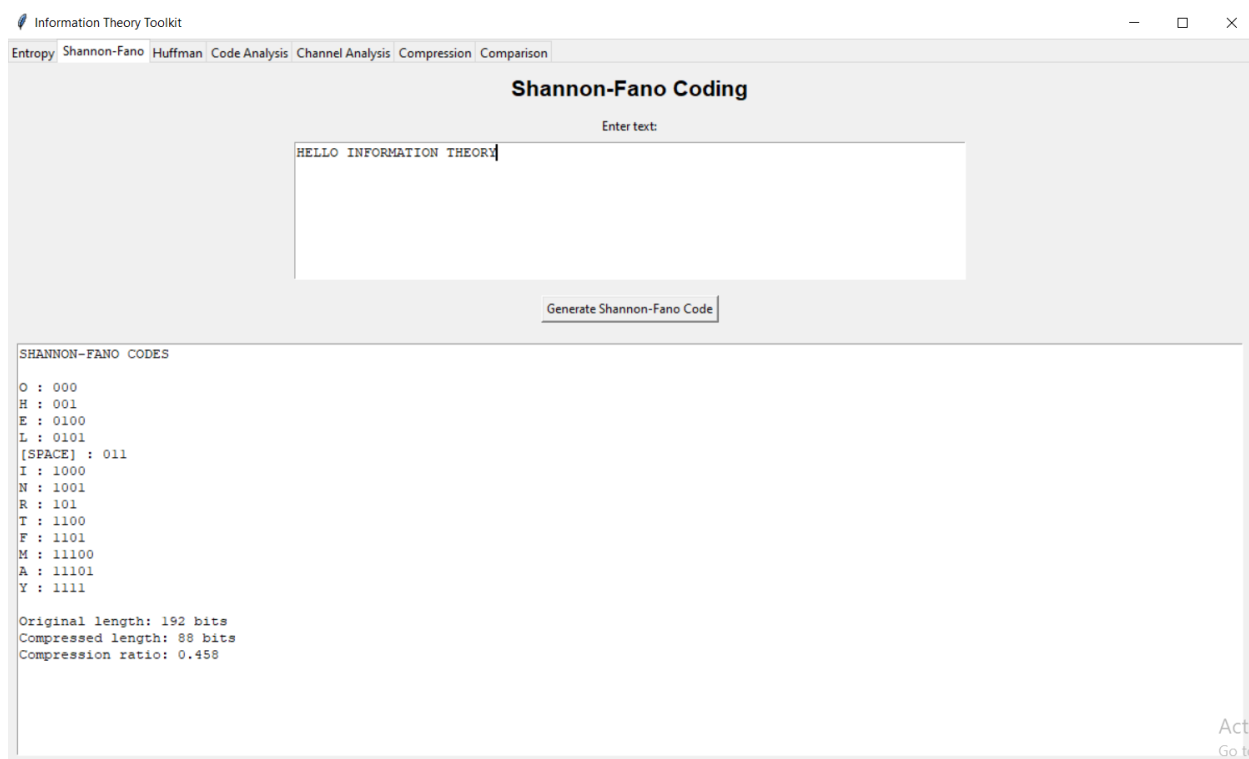
$$CR = \frac{88}{192} = 0.458$$

Добиената вредност покажува дека компресираната порака зафаќа само 45.8% од просторот потребен за оригиналната порака. Со други зборови, постигнато е намалување на должината од приближно 54.2%.

Овие резултати ја потврдуваат ефикасноста на *Huffman* алгоритмот при обработка на текстови кај кои симболите не се појавуваат со еднаква веројатност.

Добиената компресија од над 50% претставува значително подобрување во однос на фиксното 8-битно кодирање и покажува дека *Huffman* кодирањето е ефикасен метод за компресија без загуби на податоци.

5.3. Анализа на резултатите од *Shannon – Fano* кодирањето

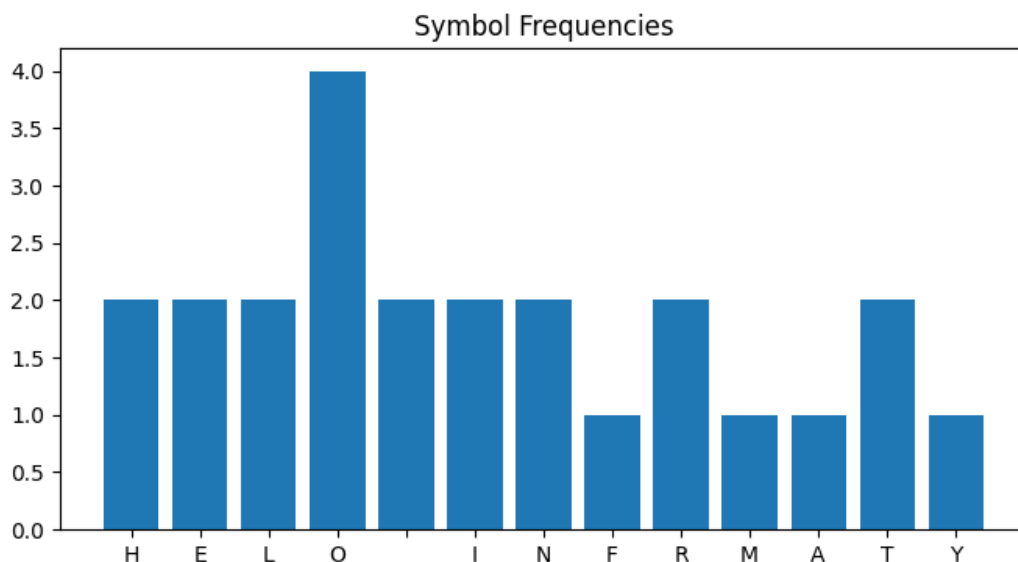


5.5. Приказ на формата за внес на порака која ќе биде кодирана со *Shannon_Fano* алгоритмот

По внесување на текст пораката во *Shannon – Fano* модулот и клик на соодветното копче, апликацијата пресметува фреквенција на појавување на симболите и врз основа на нивната распределба генерира *Shannon – Fano* кодови. За разлика од *Huffman* алгоритмот, каде кодовите се добиваат преку изградба на бинарно дрво, кај *Shannon – Fano* кодирањето симболите најпрво се подредуваат според нивната фреквенција, а потоа рекурзивно се делат во две групи со приближно еднаква вкупна веројатност.

На сликата 5.5., е прикажан резултатот од *Shannon – Fano* модулот. Во излезниот прозорец се прикажани генерираните кодови за сите симболи, како и параметрите поврзани со компресијата на пораката.

Како и кај *Huffman* кодирањето, почестите симболи добиваат пократки кодови, додека поретките симболи добиваат подолги кодови. На тој начин алгоритмот се стреми да ја намали просечната должина на кодот и да постигна компресија на податоците.



5.6. График на фреквенциите на симболите (*Shannon – Fano*)

Дополнително, апликацијата прикажува график на фреквенции на симболите, од кој може да се забележи дека одредени симболи се појавуваат значително

почесто од останатите, што претставува предуслов за успешно компресирање на пораката.

За тест пораката се добиени следните резултати:

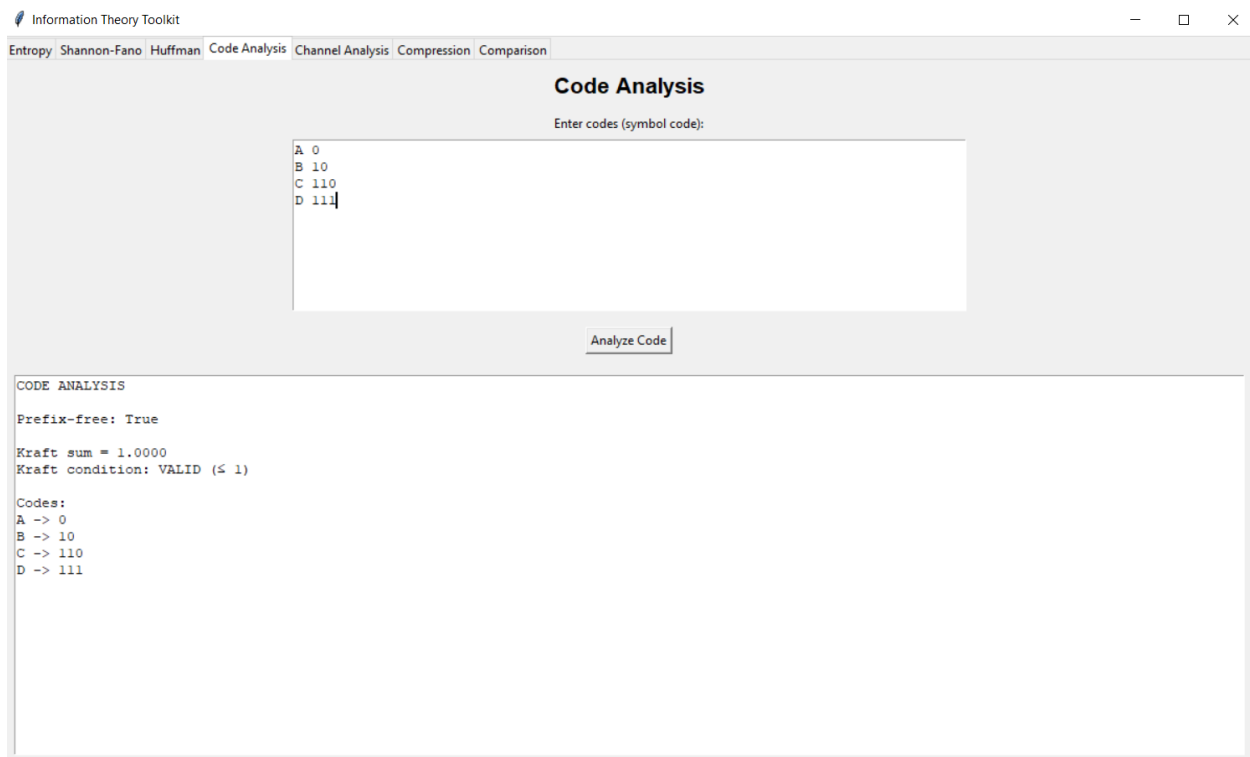
- Оригинална должина на пораката: 192 бита
- Компресирана должина на пораката: 88 бита
- Компресиски однос: 0.458

Добиените резултати покажуваат дека *Shannon – Fano* алгоритмот успешно ја намалува должината на пораката во однос на фиксното 8-битно претставување. Сепак, бидејќи поделбата на симболите се врши според приближно еднакви веројатности, добиените кодови не секогаш се оптимални. Поради тоа, во одредени случаи компресијата може да биде нешто помалку ефикасна во споредба со Huffman кодирањето.

Од аспект на Теоријата на информации, *Shannon – Fano* алгоритмот претставува историски значаен метод за конструкција на префиксни кодови и практична примена на концептите за ентропија и ефикасно претставување на информацијата. Резултатите добиени во оваа анализа ќе бидат дополнително споредени со резултатите од Huffman кодирањето во едно од следните поглавја.

5.4. Тест анализа на кодни зборови

По генерирањето на кодните зборови, потребно е да се провери дали добиениот код ги задоволува основните теоретски услови за сигурно и еднозначно декодирање. За таа цел во апликацијата е имплементиран посебен модул за анализа на кодните зборови, кој ја испитува префиксната особина на кодот и валидноста на Kraft неравенството.



5.7. Интерфејс на модулот за анализа на кодни зборови

На сликата 5.7., е прикажан изгледот на модулот за анализа на кодови. Корисникот внесува множество на кодни зборови и со клик на копчето *Analyze Code* се врши автоматска анализа на нивните својства.

За тестирање на модулот за анализа на кодни зборови беше користено следното множество на кодни зборови:

СИМБОЛ	КОДЕН ЗБОР
A	0
B	10
C	110
D	111

Table 1. Кодни зборови со кои се тестира апликацијата

По извршувањето на анализата, се добиени следниве резултати:

- Prefix-free: True
- Kraft sum = 1
- Kraft condition: valid

Добиениот резултат *Prefix-free = True* покажува дека кодот ја поседува префиксната особина. Ниту еден од кодните зборови не претставува почеток на друг коден збор, што гарантира еднозначно декодирање на пораките. Благодарение на ова својство, приемникот може точно да ги разликува поединечните симболи без потреба од дополнителни разделувачи помеѓу кодните зборови.

Дополнително е извршена проверка на Kraft неравенството. За дадениот код се добива:

$$2^{-1} + 2^{-2} + 2^{-3} + 2^{-3} = 1$$

Бидејќи добиената сума е еднаква на 1, Kraft неравенството е целосно исполнето. Вредноста 1 претставува граничен случај во кој кодното дрво е целосно искористено, односно нема неискористен коден простор.

Резултатот *Kraft condition = Valid* потврдува дека кодот може да се реализира како префиксен код и дека ги задоволува теоретските услови дефинирани во Теоријата на информации.

Од добиените резултати може да се заклучи дека анализираниот код е валиден, префиксен и погоден за сигурно кодирање и декодирање на информацијата. Истовремено, примерот претставува практична потврда на теоретските својства на префиксните кодови и Kraft неравенството разгледани во претходните поглавја.

5.5. Тест анализа на комуникациски канал

За тестирање на модулот за анализа на комуникациски канал беше дефиниран дискретен комуникациски канал преку распределбата на влезните симболи и

матрицата на условни веројатности $P(Y|X)$. Врз основа на внесените податоци апликацијата ги пресметува основните информациски параметри на каналот.

Information Theory Toolkit

Entropy Shannon-Fano Huffman Code Analysis Channel Analysis Compression Comparison

Channel Analysis

P(X):
0.7 0.3

Channel matrix (rows = X, columns = Y):

0.9	0.1
0.2	0.8

Calculate Channel

CHANNEL ANALYSIS

$H(X) = 0.8813$
 $H(Y) = 0.8932$
 $H(Y|X) = 0.5449$
 $I(X;Y) = 0.3483$
 $P(Y) = [0.69, 0.31]$

5.8. Интерфејс на модулот за анализа на комуникациски канал

За целите на тестирањето беше разгледан бинарен канал со два влезни и два излезни симболи. Распределбата на влезните симболи е зададена како:

$$P(X) = \{0.7, 0.3\}$$

Додека матрицата на условни веројатности е:

$$P(Y|X) = \begin{bmatrix} 0.9 & 0.1 \\ 0.2 & 0.8 \end{bmatrix}$$

Врз основа на овие податоци апликацијата ги пресмета следните информациски параметри:

- $H(X) = 0.8813$ бита

- $H(Y) = 0.8932$ бита
- $H(Y|X) = 0.5449$ бита
- $I(X, Y) = 0.3483$ бита
- $P(Y) = \{0.69, 0.31\}$

Ентропијата на влезот $H(X)=0.8813$ бита ја претставува просечната количина на информација која ја содржи изворот пред преносот. Бидејќи влезните симболи не се еднакво веројатни, ентропијата е помала од максималната вредност од 1 бит која би се добила кај рамномерна распределба.

Пресметаната распределба на излезните симболи е:

$$P(Y) = \{0.69, 0.31\}$$

Од каде се добива ентропијата на излезот од 0.8932 бита. Вредноста е многу блиска до ентропијата на влезот, што покажува дека структурата на информацијата во голема мера е зачувана по преносот низ каналот.

Условната ентропија изнесува:

$$H(Y|X) = 0.5449$$

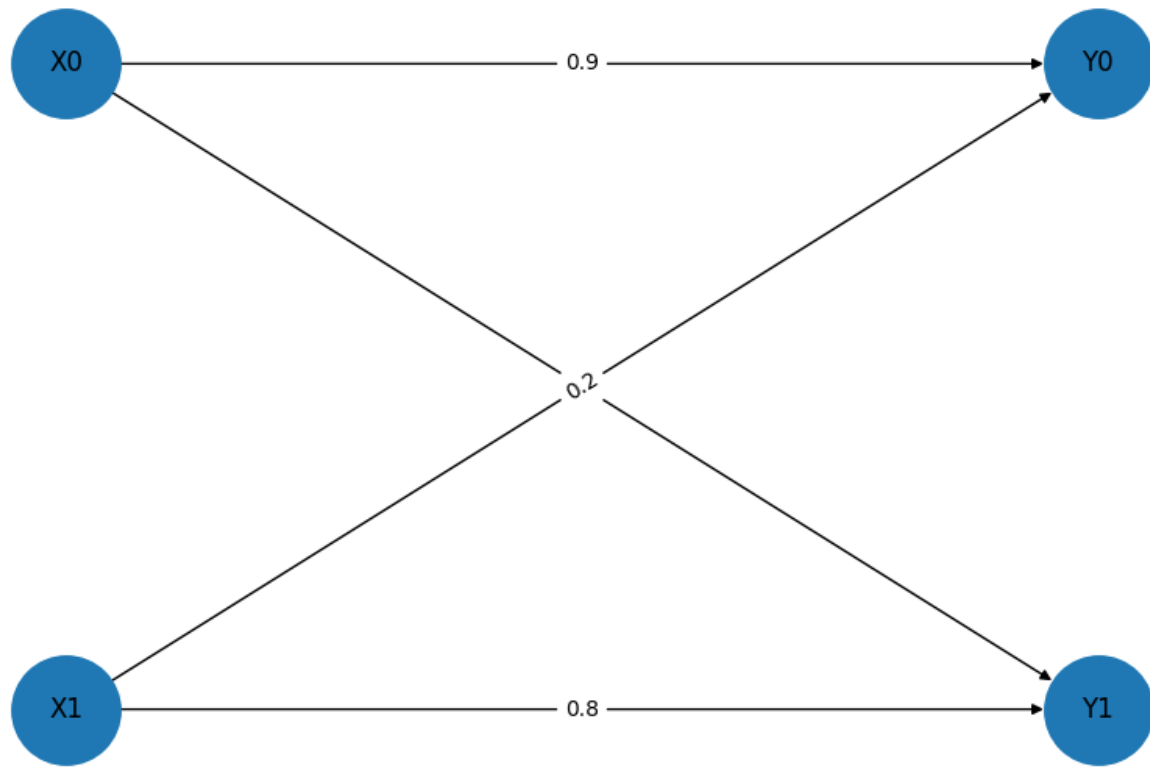
Оваа величина ја мери неизвесноста која останува на излезот дури и кога влезниот симбол е познат. Нејзината вредност покажува дека каналот не е идеален и дека постои одредено ниво на шум, поради кое не секој влезен симбол секогаш произведува ист излезен симбол.

Количината на успешно пренесена информација е определена преку заемната информација:

$$I(X, Y) = H(Y) - H(Y|X) = 0.8932 - 0.5449 = 0.3483 \text{ бита}$$

Добиената вредност покажува дека во просек околу 0.35 бита информација по симбол успешно се пренесуваат низ каналот. Останатиот дел од информацијата се губи поради неизвесноста внесена во каналот.

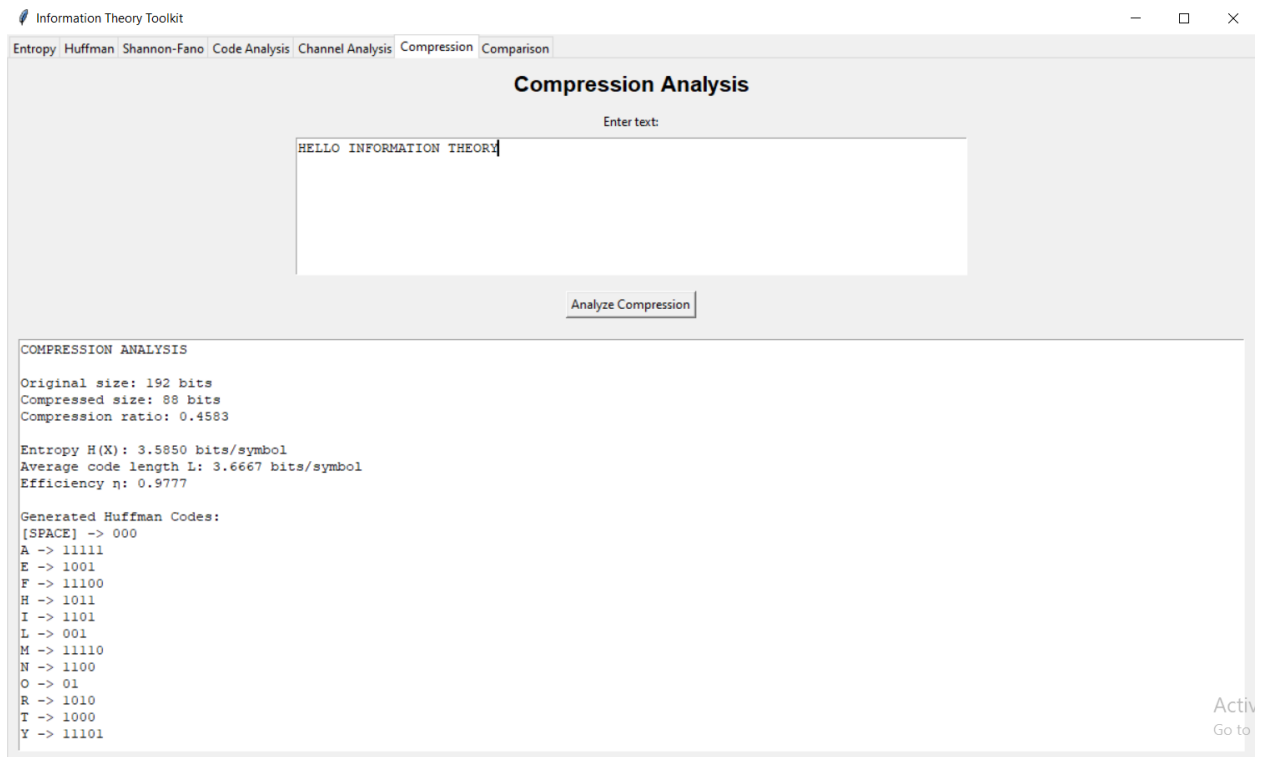
Од анализата може да се заклучи дека каналот обезбедува релативно сигурен пренос на информација, но не е целосно без шум. Поради постоењето на ненулта условна ентропија, дел од информацијата се нарушува при преносот. Сепак, позитивната вредност на заемната информација покажува дека постои статистичка зависност помеѓу влезот и излезот, односно приемниот сигнал се уште содржи коризна информација за испратената порака.



5.9. Графички приказ на тестираниот бинарен комуникациски канал

На сликата 5.8., е претставен графички приказ на тестираниот канал, генериран од апликацијата. На приказот се забележуваат веројатностите на премин помеѓу влезните и излезните симболи, што овозможува визуелна интерпретација на процесот на пренос и влијанието на шумот врз комуникацијата.

5.6. Тест анализа на модулот за компресија



5.10. Интерфејс на модулот за компресија

За тестирање на модулот за компресија беше искористена истата тест порака: *HELLO INFORMATION THEORY*

При анализата, апликацијата најпрво ја пресметува фреквенцијата на појавување на секој симбол во пораката, по што се конструира *Huffman* дрво и се генерираат соодветните кодни зборови. Врз основа на добиените кодови се формира компресираната бинарна претстава на пораката и се пресметуваат основните параметри за оценување на ефикасноста на компресијата.

По извршената анализа се добиени следните резултати:

- Оригинална големина на пораката: 192 бита
- Компресирана големина на пораката: 88 бита
- Однос на компресија: 0.4583
- Ентропија на изворот: 3.5850 бита/симбол

- Просечна должина на кодот: 3.6667 бита/симбол
- Ефикасност на кодот: 0.9777

Оригиналната големина од 192 бита е добиена под претпоставка дека секој знак се претставува со 8 бита. По примената на *Huffman* кодирањето, должината на пораката се намалува на само 88 бита, што претставува значително намалување на потребниот простор за складирање или пренос.

Односот на компресија изнесува:

$$CR = \frac{88}{192} = 0.4583$$

Ова значи дека компресираната порака зафаќа само 45.83 од просторот потребен за оригиналната порака. Соодветно, постигнато е намалување на должината од приближно 54.17%, што претставува значајна компресија без губење на информација.

Ентропијата на изворот изнесува 3.5850 бита по симбол и ја претставува теоретската граница под која неможе да се намали просечната должина на кодот кај безгубиточно кодирање. Од друга страна, добиената просечна должина на *Huffman* кодот изнесува 3.6667 бита по симбол.

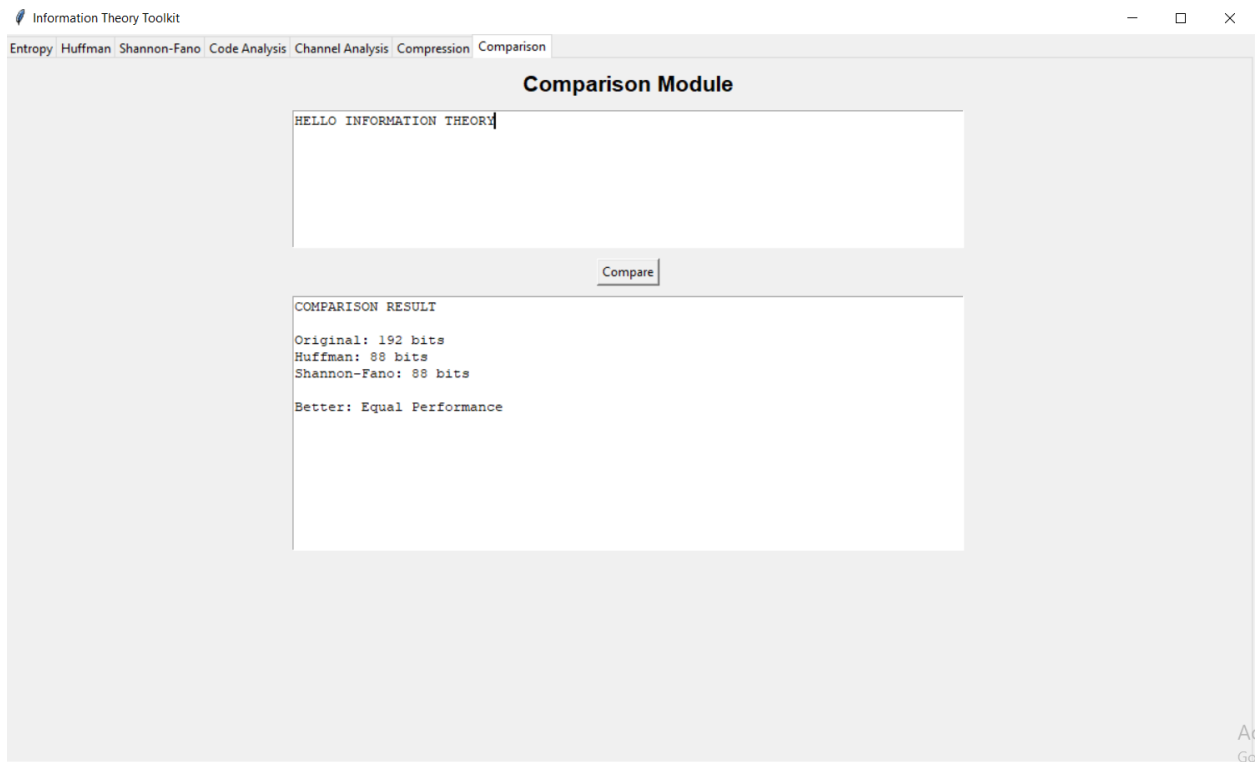
Разликата помеѓу овие две вредности е многу мала:

$$3.6667 - 3.5850 = 0.0817 \text{ бита/симбол}$$

што покажува дека генерираниот код е многу близок до теоретски оптималното решение.

Ефикасноста на кодот изнесува 0.9777, односно приближно 97.77%. Оваа вредност покажува колку просечната должина на кодот е блиска до ентропијата на изворот. Бидејќи ефикасноста е многу блиску до 1, може да се заклучи дека *Huffman* алгоритмот исклучително успешно ја искористил статистичката структура на пораката.

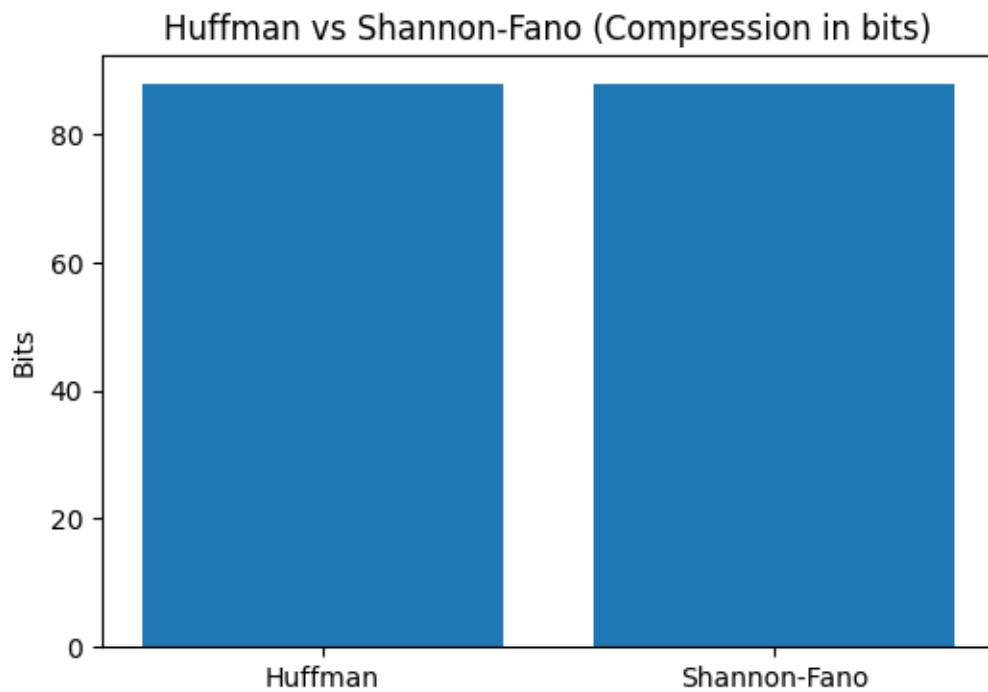
5.7. Тест анализа на модулот за споредба меѓу двата анализирани алгоритми



5.11. Интерфејс на модулот за споредба на перформансите на алгоритмите

На сликата 5.11., се прикажани резултатите од извршувањето на модулот за споредба меѓу двата досега анализирани алгоритми. Модулот за споредба ја обработува истата тест порака, по што ја пресметува должината на добиените кодирани пораки и ги споредува резултатите.

Од добиените резултати може да се забележи дека двата алгоритми постигнуваат идентична компресија на анализираната порака. Во двата случаи должината на кодираната порака изнесува 88 бита, што претставува намалување од повеќе од 50% во однос на оригиналната должина.



5.12. графички приказ на добиените резултати од споредбата меѓу *Huffman* и *Shannon – Fano* алгоритмите

Графичкиот приказ дополнително потврдува дека добиените резултати се практично еднакви. Иако *Huffman* алгоритмот теоретски се смета за оптимален алгоритам за конструкција на префиксни кодови, кај одредени распределби на симболите *Shannon – Fano* алгоритмот може да постигне исти резултати.

Во конкретниот пример не постои разлика во ефикасноста на двата алгоритми, бидејќи компресираната должина е идентична. Поради тоа не може да се заклучи дека еден од алгоритмите е подобар од другиот врз основа на овој тест случај.

6. Можности за понатамошен развој на апликацијата

Развиената апликација успешно ги демонстрира основните концепти од Теоријата на информации, вклучувајќи пресметка на ентропија, *Huffman* и *Shannon – Fano* кодирање, анализа на кодови, анализа на комуникациски канали, компресија на податоци и споредба на различни алгоритми за кодирање. Иако апликацијата ги

исполнува поставените цели на проектот, постојат повеќе можности за нејзино понатамошно унапредување.

Една од можностите е зголемување на интерактивноста на системот преку овозможување на корисникот самостојно да ја дефинира големината на азбуката и бројот на симболи кои се анализираат. На тој начин би можеле да се обработуваат посложени информациски извори со поголем број симболи и поразновидни распределби на веројатностите.

Дополнително, модулот за анализа на комуникациски канал би можел да се прошири со поддршка за канали со произволен број влезни и излезни симболи. Наместо фиксни матрици со две редици и две колони, корисникот би можел сам да ја дефинира димензијата на матрицата на условни веројатности, што би овозможило анализа на значително покомплексни комуникациски системи.

Во насока на подобрување на визуелната презентација, покрај постојните графици и дијаграми би можеле да се додадат анимации кои би го прикажувале процесот на конструирање на *Huffman* дрвото, формирањето на *Shannon – Fano* поделбите или преносот на информацијата низ комуникацискиот канал. Ваквите визуелизации би придонеле за полесно разбирање на алгоритмите нивната работа.

Апликацијата би можела да се прошири и со имплементација на дополнителни алгоритми за кодирање и компресија, како што се *Arithmetic Coding*, *Lempel – Ziv* (LZ77, LZ78), *Run – Length Encoding (RLE)* и други методи кои имаат значајна примена во современите системи за обработка и пренос на информации.

Друга можност е воведување на подетална класификација на анализите, при што посебно би се обработувале карактеристиките на информацискиот извор, својствата на кодовите и карактеристиките на комуникацискиот канал. Со тоа би се добила појасна организација на содржините и подлабока анализа на секој сегмент од Теоријата на информации.

Исто така, би можеле да се имплементираат дополнителни пресметки како капацитет на канал, релативна ентропија, условна заемна информација, анализа на шум и други напредни параметри кои се користат во комуникациските канали.

Сепак, за целите на овој проект имплементираната функционалност е сосема доволна за демонстрација на основните концепти од Теоријата на информации. Апликацијата овозможува практична примена на теоретските модели изулувани во наставата и претставува солидна основа за идни надградби и понатамошен развој.

7. Заклучок

Теоријата на информации претставува една од најзначајните научни дисциплини на современото информатичко и комуникациско општество. Таа обезбедува математичка основа за мерење, претставување, пренос и обработка на информацијата, независно од нејзината физичка природа или конкретна примена.

Преку концептите како ентропија, кодирање, компресија и анализа на комуникациски канал, Теоријата на информации овозможува подобро разбирање на начинот на кој информацијата се создава, складира и пренесува. Нејзините принципи претставуваат темел на современите компјутерски системи, телекомуникации, мултимедијални технологии, криптографија, вештачка интелигенција и многу други области.

И покрај тоа што е развиена пред повеќе од седум децении, Теоријата на информации и денес останува актуелна и незаменлива научна рамка за развој на ефикасни системи за обработка и размена на податоци. Нејзините идеи продолжуваат да имаат значајно влијание врз развојот на современите технологии и претставуваат основа за идните истражувања во областа на информациските и комуникациски науки.

8. Користена литература

1. **„Теорија на информации – скрипта со предавања“** - доцент д-р Наташа Стојковиќ, проф. д-р Зоран Утковски, асистент - докторанд д-р Марија Митева, асистент - докторанд д-р Елена Карамазова
2. **„Теорија на информации – практикум со вежби“** - асистент - докторанд д-р Марија Митева, доцент д-р Наташа Стојковиќ, проф. Д-р Зоран Утковски